自然语言处理

第6讲：词向量及神经语言模型

杨沐昀

哈尔滨工业大学计算学部
语言技术研究中心

yangmuyun@hit.edu.cn

主要内容

- 文本的向量表示
- 神经网络语言模型
- 预训练语言模型

独热表示

- 文本的向量化表示
  - 将不同粒度的文本映射到向量空间
  - 对文本的处理可以转化为对向量的操作
- 独热表示（One-hot Representation）
  - 早期的文本向量表示方法：每个维度表示某个单词是否在该文中出现：
    - “计算机”表示为 [1, 0, 0, ..., 0]
    - “电脑”　表示为 [0, 1, 0, ..., 0]
    - “冰激凌”表示为 [0, 0, 1, ..., 0]
- 问题
  - 维度和词表的大小一致
  - 存在表示稀疏性的问题
  - 无法表示单词之间的语义相似度

单词的分布式表示

- 单词的分布式表示（Word Distributed Representation）：
  - 将单词映射到一个连续向量空间中，例如：
    - “计算机”表示为 [0.16, 0.19, -0.28, ..., 0.87]
    - “电脑”　表示为 [0.20, 0.17, -0.21, ..., 0.97]
    - “冰激凌”表示为 [-0.90, 0.72, 0.65, ..., 0.06]
  - 向量通常是高维的
  - 能够捕捉单词之间的语义相似性
- 分布式表示的目标是建立单词嵌入矩阵 $W \in R^{|V|*d}$，其中矩阵的每一行对应一个单词，为单词的向量表示，即词向量 （Word Embedding）。

单词的分布式表示

- 与独热表示相比，分布式表示可以编码不同单词之间的语义关联。
  - 采用独热表示，（“计算机”，“电脑”）或者（“计算机”，“冰激凌”）的向量均为正交，其相似度相同。
  - 采用分布式表示可以使得“计算机”和“电脑”在大多数维度上相近，这样“计算机”和“电脑”的向量之间的距离可以远小于“计算机”和“冰激凌”之间的距离，从而通过向量的距离度量体现出单词之间的相似程度以及语义关系。

分布式假设

- 大部分单词分布式表示方法都遵从分布式假设：
  - 出现在相同上下文中的单词往往具有相似的语义
  - 单词的意义可以通过其在数据集中出现的上下文来推断

- 示例：
  - 狮子是非洲的草原上的顶级猎食者，经常捕猎羚羊等动物。
  - 非洲的草原上，豹子是非常强大的猎食者，擅长捕猎羚羊和其他小型动物。
  - 这两个句子中的“狮子”和“豹子”有相似的上下文，根据分布式假设，可以推测这两个单词的语义是相似的，且它们的向量表示也应该相近。

单词分布式表示学习方法

- 常见的单词分布式表示学习方法：
  - 基于共现矩阵奇异值分解的词向量模型
  - 基于上下文单词预测的词向量模型
  - 全局向量模型
  - 基于字节对编码的子词表示模型

基于奇异值分解的词向量模型

- 共现矩阵：
  - 通过统计单词在数据集中的共现频率，构造一个反映单词之间关系的高维矩阵，并在此基础上通过矩阵分解降低维度从而获得低维稠密的词向量。

- 潜在语义分析 （Latent Semantic Analysis, LSA）模型采用奇异值分解方法 （Singular Value Decomposition, SVD），将单词文档共现矩阵 （Term-Document Co-occurrence Matrix） 或单词上下文共现矩阵 （Window based Co-Occurrence Matrix） 转换为单词向量表示。

基于奇异值分解的词向量模型

- 步骤：
  - 1）构建共现矩阵
  - *2）矩阵元素调整（可选）*
  - *3）降维处理*

代码 6-1：共现矩阵统计算法

输入：训练数据集 D，上下文窗口大小 n

输出：词表 $\mathbb{V}$，共现矩阵 A

$\mathbb{V} = \text{unique}(D)$ // 由数据集统计出现的词表

$\mathbf{A}_{ij} = 0, \forall i, j \in [1, ..., |\mathbb{V}|]$ // 初始化共现矩阵

foreach $s \in D$ do

  $w_1, ..., w_N = s$ // 取训练数据集中的句子,句子由 N 个词构成

  $id_i = \mathbb{V} \cdot \text{index}(w_i), \forall i \in [1, ..., N]$ // 将句子中的单词转化为词表下标

  for $i = 1$ to N do

    for $j = \max(1, i-n)$ to $\min(N, i+n), j \neq i$ do

      $\mathbf{A}_{id_i, id_j} = \mathbf{A}_{id_i, id_j} + 1$ // 共现计数

    end

  end

end

return $\mathbb{V}, A$

基于奇异值分解的词向量模型

- 共现矩阵的每一行可以直接视为对应词的向量表示,能够反映词语之间的相似性。

- 存在的问题：
  - 1）共现频率是否可以作为衡量单词语义关系的合适手段?
  - 2）向量维度与词汇表的大小一致，有大量的零元素，存在表示稀疏性的问题。

基于奇异值分解的词向量模型

- 步骤：
  - *1）构建共现矩阵*
  - 2）矩阵元素调整（可选）
  - *3）降维处理*

- 矩阵元素调整可解决第一个问题，但并不是必需的。

- 常见的操作：对共现次数进行平滑或加权处理（如计算PMI，Pointwise Mutual Information），从而避免频率偏差。

基于奇异值分解的词向量模型

- 步骤：
  - *1）构建共现矩阵*
  - *2）矩阵元素调整（可选）*
  - 3）降维处理

- 降维处理可解决第二个问题，是单词分布式表示方法的关键所在。

- 奇异值分解方法将共现矩阵分解为低阶近似矩阵，从而为每个单词提供低维表示。

基于奇异值分解的词向量模型

- 步骤：
  - *1）构建共现矩阵*
  - *2）矩阵元素调整（可选）*
  - 3）降维处理

- 对于矩阵$A \epsilon R^{m \times n}$，通过奇异值分解可将其分解为$A = U \Sigma V^T$，其中$U \epsilon R^{m \times m}$和$V \epsilon R^{n \times n}$是酉矩阵，$\Sigma \epsilon R^{m \times n}$在主对角线之外的元素为0。

- 奇异向量的顺序均按奇异值由大及小顺序排列。矩阵Σ对角线上的奇异值元素均为非负实数，且按由大及小顺序排列。

基于奇异值分解的词向量模型

- 步骤：
  - *1）构建共现矩阵*
  - *2）矩阵元素调整（可选）*
  - 3）降维处理

- 矩阵奇异值的大小分布极不均匀，前10%奇异值之和可以占到全部奇异值之和的90%以上。

- 可以在奇异值矩阵中只保留最大的d个奇异值，同时在两侧矩阵中仅保留对应的分量，对矩阵A进行低秩近似。

基于上下文单词预测的词向量模型

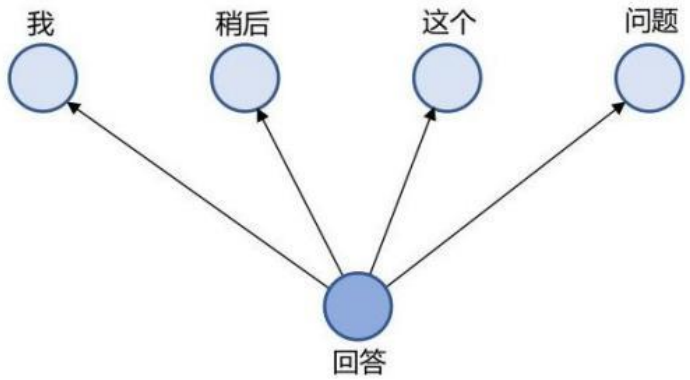
- Word2Vec:
  - 基于上下文单词预测的模型
  - 自监督的方式
  - 从大量无标注文本中通过训练简化的神经网络得到词向量

- Word2Vec是自然语言处理中的经典方法，为后续的词嵌入模型（如 GloVe 和 FastText）奠定了基础。

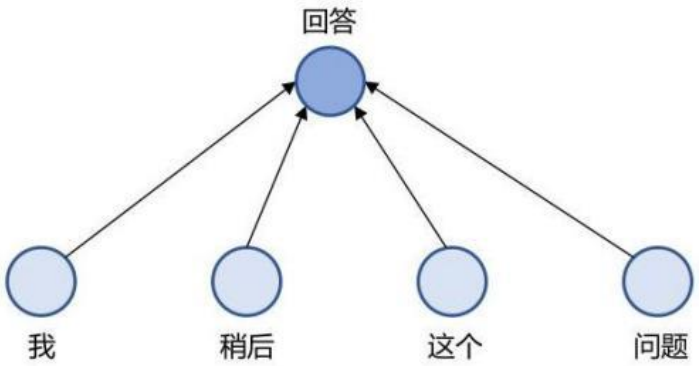
- 而且，它训练出来的词向量被证明捕获到了词间的语义关系，如“国王 - 男性 + 女性 ≈ 皇后”，这启发了一系列相关的研究工作。

基于上下文单词预测的词向量模型

- Word2Vec主要有以下两种：
  - Skip-gram模型
  - 连续词袋模型（Continuous Bag of Words，CBoW）



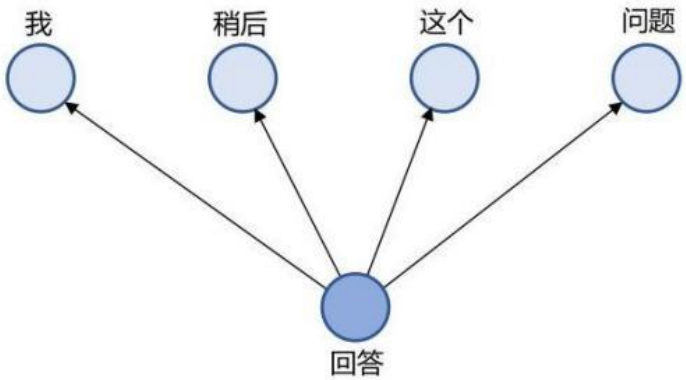
Skip-Gram 模型



CBoW 模型

基于上下文单词预测的词向量模型

- Skip-Gram模型：
  - 根据给定词可以预测其上下文窗口内的其他词
  - 通过最大化上下文词在中心词已知条件下的条件概率来进行训练



Skip-Gram 模型

基于上下文单词预测的词向量模型

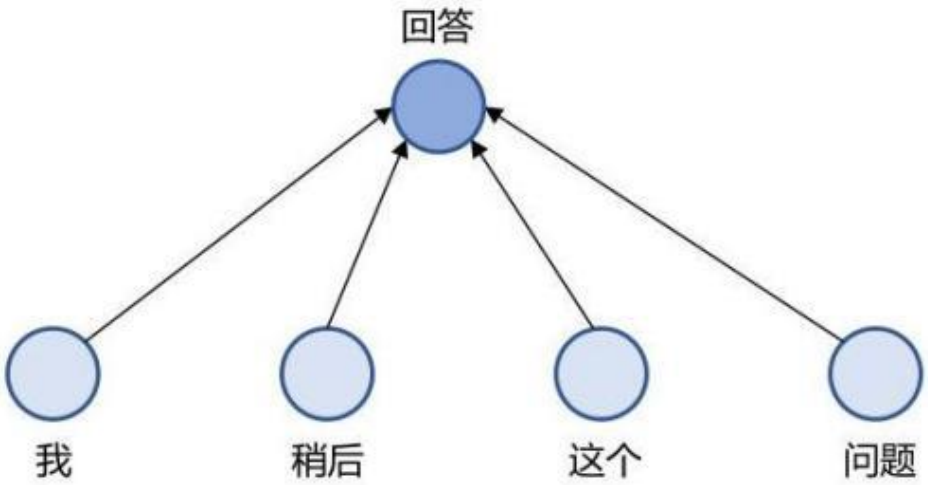
- Skip-Gram模型：
  - 损失函数选择负对数概率作为优化目标

$$\mathcal{C}\left(w_1 \ldots w_T\right) = -\sum_{t=1}^{T} \sum_{-n \le i \le n, i \neq 0} \log P\left(w_{t+i} | w_t\right)$$

  - 其中，w_t是位置t的中心词，w_{t+i}是上下文窗口内每个位置的上下文词，T是文本序列的长度。$\mathrm{P}(w_t | w_{t+i})$条件概率通过训练上下文词和中心词的词嵌入参数矩阵来近似估计。

基于上下文单词预测的词向量模型

- CBoW 模型：
  - 文本中的词可以通过其在上下文的其他单词推测出来
  - 通过最大化中心词在给定上下文下的条件概率来进行训练



CBoW 模型

基于上下文单词预测的词向量模型

- CBoW 模型：
  - 采用负对数概率损失函数作为优化目标

$$\mathcal{L}\left(w_1 \ldots w_T\right) = -\sum_{t=1}^{T} \log P\left(w_t | w_{t-n}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+n}\right)$$

  - 通过学习上下文词向量矩阵和中心词词向量矩阵来优化目标函数

基于上下文单词预测的词向量模型

- 实际应用中，由于词汇表通常包含数万到数十万个单词，Skip-Gram 和 CBoW 模型在使用 Softmax 计算上下文词和中心词的出现概率并进行梯度更新时，会产生巨大的计算开销。

- 降低计算负担方法：
  - 负采样
  - 层次Softmax

- 结合Skip-Gram模型介绍负采样方法

基于上下文单词预测的词向量模型

- 负采样（Negative Sampling）
  - 将目标函数中的全词表相似度计算改为目标词与 K 个负例词的相似度计算
  - 其中 K 是远小于词表大小的超参数
  - 训练的计算开销与词表大小无关，而仅与超参数 K 相关

基于上下文单词预测的词向量模型

- 负采样（Negative Sampling）
  - 首先将词向量的相似度与词在上下文窗口中出现的概率相关联

$$P\left(D=1|w_o,w_c\right)=\sigma\left(\mathbf{u}_o^T\mathbf{v}_c\right)$$

$$P\left(D=0|w_o,w_c\right)=1-P\left(D=1|w_o,w_c\right)$$

  - 其中，$D=1$或$D=0$表示这组上下文词-中心词是否来自训练语料，即训练语料中单词w_o是否出现在w_c的上下文窗口中。σ表示Sigmoid函数，u_o是w_o用作上下文词的表示，v_c是w_c用作中心词的表示。

基于上下文单词预测的词向量模型

- 负采样（Negative Sampling）
  - 对于每一对上下文词-中心词，原始的 Skip-Gram 模型仅优化正样本$P(D=1|w_o,w_c)$，而在负采样模型中，从原始词分布$P(\mathrm{w})$中采样K个未出现在上下文窗口中的词$w_1,\ldots,w_K$作为负样本，并额外优化$P(D=0|w_i,w_c)$，形式化表示为:

$$P\left(w_o|w_c\right)=P\left(D=1|w_o,w_c\right)\prod_{i=1}^{K}P\left(D=0|w_i,w_c\right)$$

  - 将上述公式代替原概率项估计，可得到负采样下的目标函数

全局向量模型

- GloVe（Global Vectors for Word Representation）模型:
  - 结合了潜在语义分析（LSA）和Word2vec两种方法的优点
  - 既考虑局部上下文信息，又利用整个数据集的全局共现统计数据
- 与LSA类似之处:
  - 基于词共现矩阵进行分解
- 与LSA不同之处:
  - GloVe 通过优化共现概率的加权损失函数，直接建模词对之间的共现关系
  - LSA 更侧重于通过矩阵分解来提取潜在语义结构

全局向量模型

- GloVe（Global Vectors for Word Representation）模型:
  - 在共现计数方法基础上进一步引入了中心词与上下文词之间的距离因素
  - 距离较近的中心词-上下文词对在共现次数中占据更大的权重
  - $C_{ij} = \sum d^{-1}(w_i, w_j)$
  - 其中w_i, w_j表示词表中下标为i和j的单词，在它们的每次共现中，使用$d(w_i, w_j)$表示单词w_i,w_j之间的距离
  - 矩阵中单词w_j出现在单词w_i上下文的共现概率为$p_{ij} = P(w_j|w_i) = \frac{C_{ij}}{\sum_{j=1}^{|V|} C_{ij}}$

全局向量模型

- GloVe（Global Vectors for Word Representation）模型
  - 记$\hat{p}_{ij} = P(w_j|w_i) = \frac{C_{ij}}{\sum_{j=1}^{|V|} C_{ij}}$为$w_j$出现在$w_i$上下文范围内的预测概率，即$p_{ij}$的预测值
  - 利用词向量对“词-上下文”共现矩阵进行预测（或回归）

$$\mathcal{C} = -\sum_{i=1}^{|\mathbb{V}|}\sum_{j=1}^{|\mathbb{V}|} h_{ij}\left(\mathbf{u}_j^T\mathbf{v}_i - \log C_{ij} + b_i + c_j\right)^2$$

  - 考虑到共现矩阵是对称的，GloVe 模型中优化得到的中心词向量和上下文词向量理论上应该相等
  - 实际应用中，由于权重的随机初始化差异，同一词的中心词向量和上下文词向量可能并不完全相同
  - 通常将这两个向量的和作为最终的词向量输出

基于字节对编码的子词表示模型

- 未登录词
  - 词表中不存在的词
  - 在词表中加入一个表示未知词的标识“[UNK]”（unknown）

- 上述词表示模型都依赖于预先定义的词表V，只能处理词表中存在的词
  - 遇到词表外的未登录词，模型无法为其生成对应的词表示，只能为这些词分配一个默认的通用表示
  - 将其向量作为词表示矩阵的一部分进行训练，同时通过某些机制来更新 “[UNK]” 向量的参数
  - 所有未登录词都会使用 “[UNK]” 向量作为其表示

基于字节对编码的子词表示模型

- 基于固定词表的词表示模型对词表大小的选择较为敏感:
  - 词表过小时，未登录词的比例较高，影响模型性能
  - 词表过大时，大量低频词被包含在内，这些低频词的词向量难以得到充分学习

- 理想的词表示模型应能覆盖绝大多数输入词，并避免因词表过大而导致的数据稀疏问题

基于字节对编码的子词表示模型

- 为了缓解未登录词的问题，一些研究通过利用亚词级别的信息来构建词表示向量：
  - 方法一：建立字符级别的表示，组合字符向量来获得每个单词的表示
  - 存在的问题：词根、词缀等构词模式通常跨越多个字符，基于字符的表示方法往往难以捕捉这些跨度较大的模式

基于字节对编码的子词表示模型

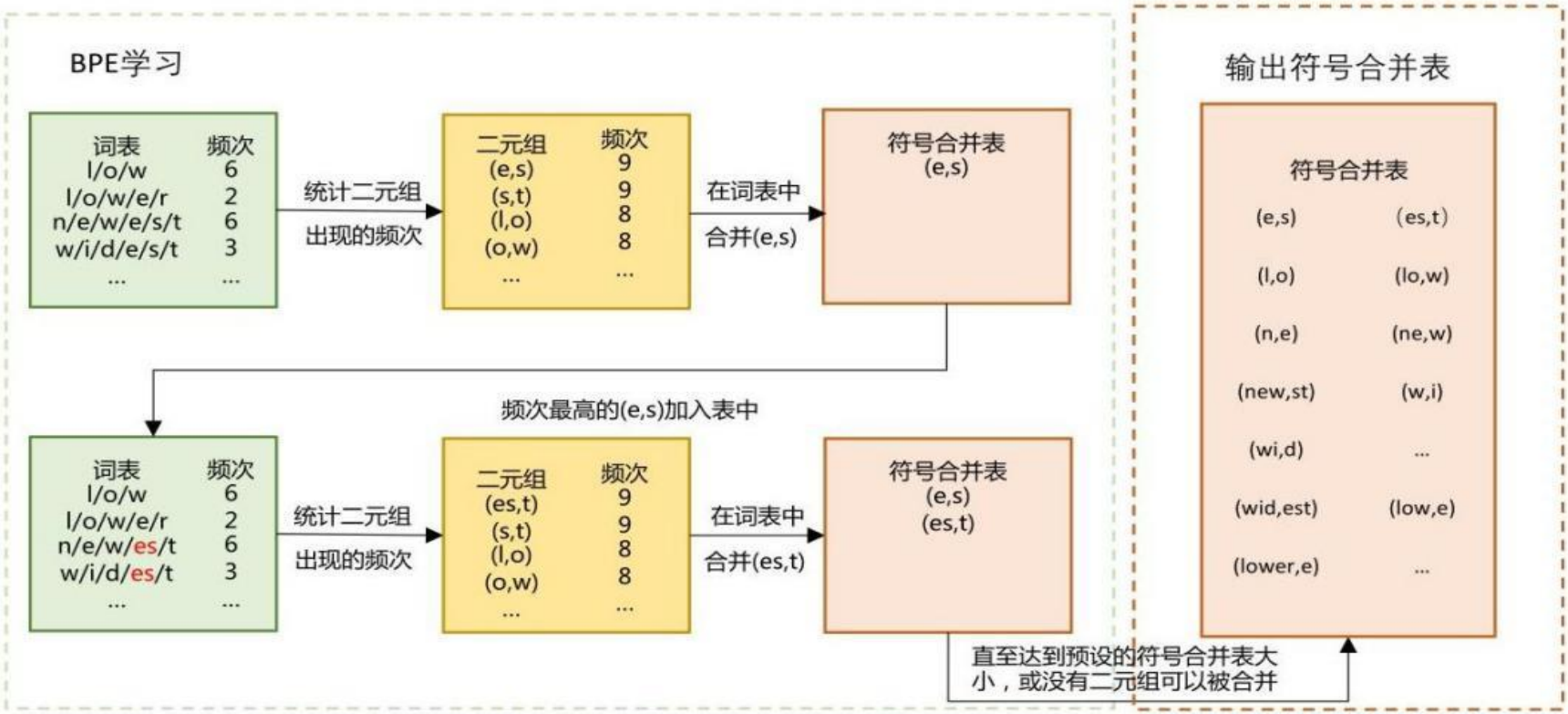
- 为了缓解未登录词的问题，一些研究通过利用亚词级别的信息来构建词表示向量：
  - 方法二：子词表示模型
    - 维护一个子词词表
    - 词表中既包含完整的单词，也包含像“c”、“re”、“ing”等单词的部分信息，称为子词
    - 在处理输入词序列时，子词表示模型会将每个词拆分为词表中的子词
      - 例如，单词“reborn”可以拆分为“re”和“born”
    - 需要计算某个单词或短语的表示时，可以将对应的子词表示合并为所需的词表示

基于字节对编码的子词表示模型

- 字节对编码（Byte Pair Encoding, BPE）模型：
  - 一种常见的子词表示方法
  - 词表包含了常见的单词以及高频出现的子词
  - 在实际应用中：
    - 常见词通常直接位于BPE词表中
    - 罕见词被分解为若干个已包含在BPE词表中的子词，显著降低未登录词的比例

基于字节对编码的子词表示模型

- 字节对编码（Byte Pair Encoding, BPE）模型：
  - 两个主要步骤：（1）确定子词词表；（2）将全词切分为子词，或者将子词合并为全词



基于字节对编码的子词表示模型

- 字节对编码（Byte Pair Encoding, BPE）模型：
  - 两个主要步骤：（1）确定子词词表；*（2）将全词切分为子词，或者将子词合并为全词*
    - 确定数据集中所有单词的词表和词频
    - 将每个单词切分为单个字符的序列
    - 在每个单词的结尾添加符号“</w>”
      - 例如，单词“low”将被切分为“l o w </w>”，切分出的元素称为字节

基于字节对编码的子词表示模型

- 字节对编码（Byte Pair Encoding, BPE）模型：
  - 两个主要步骤：（1）确定子词词表；*（2）将全词切分为子词，或者将子词合并为全词*
    - 根据每对相邻字节的词频，统计并合并出现频率最高的字节对，将其作为新的子词添加到词表中，并在所有单词中合并该字节对为新的子词
      - 比如，在第一次迭代中，若字节对“e”和“s”出现频率最高，则将“es”作为子词加入词表，并将所有序列中相邻的“e”和“s”合并为“es”
    - 持续进行上述过程，直到 BPE 子词词表的大小达到预设值，或者不再有可以合并的字节对

基于字节对编码的子词表示模型

- 字节对编码（Byte Pair Encoding, BPE）模型：
  - 两个主要步骤：*（1）确定子词词表；*（2）将全词切分为子词，或者将子词合并为全词
  - 对输入词序列中未出现在词表中的单词进行切分
  - 按照从长到短的顺序遍历词表中的子词，尝试将每个子词与当前单词或尚未完全切分的部分进行匹配，并将单词切分为该子词和剩余部分的序列
    - 例如，对于单词“lowest</w>”，首先匹配子词“est</w>”，将其切分为“low”和“est</w>”，然后再匹配子词“low”，最终确定其切分结果为“low”和“est</w>”
  - 将词序列中的单词分解为已知的子词，解决未登录词的问题

基于字节对编码的子词表示模型

- 字节对编码（Byte Pair Encoding, BPE）模型：
  - 两个主要步骤：*（1）确定子词词表；*（2）将全词切分为子词，或者将子词合并为全词
  - 对于切分得到的子词序列，为每个子词查询其对应的子词表示，并将这些表示组合成一个子词表示序列
  - 如果遇到未登录的子词，则采用类似处理未登录词的方法，为其分配一个通用的表示，最终得到输入的完整子词表示序列

基于字节对编码的子词表示模型

- 字节对编码（Byte Pair Encoding, BPE）模型：
  - 两个主要步骤：*（1）确定子词词表；*（2）将全词切分为子词，或者将子词合并为全词
  - 对于采用子词表示模型的自然语言处理系统，其输出也会是子词序列
    - 例如：机器翻译系统
  - 将原始输出恢复为词序列：
    - 根据终结符“</w>”的位置来确定每个单词的范围
    - 将这些子词合并为对应的单词
    - 将最终的词序列作为系统的输出结果

单词分布式表示的评价与应用

- 词向量模型的定量评估方法：
  - 内部评价（Intrinsic Evaluation）
    - 依赖于设计特殊的辅助任务
    - 测试词向量是否能够有效地表达特定的语义特征，例如词义相关性或类比性，并通过返回一个分数来衡量词向量的质量
  - 外部评价（Extrinsic Evaluation）
    - 基于实际应用任务
    - 将词向量作为输入，评估不同词向量模型在该任务中的表现，以选择最适合该任务的词向量模型

单词分布式表示的评价与应用

- 在对词向量模型的综合性能进行评估时，通常会采用内部评估方法。
- 因为内部评估方法使用的辅助任务相对简单，计算速度也更快。
- 外部评估方法不仅涉及词向量模型，还需要考虑下游任务的模型。当系统整体性能不达预期时，问题可能出在词向量模型、任务模型，或者两者的交互作用上，因此外部评估难以准确地指导词向量模型的选择和优化。

单词分布式表示的评价与应用

- 常见的内部评估方法：
  - 词义相关性
  - *词语类比性*

- 词义相关性评估任务：检验词向量在表达词义相关性方面的能力

- 词向量应当在低维语义空间中紧凑且连续地分布，因此需要采用一种简洁且易于计算的相似度度量方法，使得词向量之间的相似度能够有效地反映词语间的语义相关性。

单词分布式表示的评价与应用

- 常见的内部评估方法：
  - 词义相关性
  - *词语类比性*

- 具体地，对于单词w_i, w_j及其词向量v_i, v_j，简单地使用余弦相似度作为词义相似度的度量：

$$\text{sim}\left(w_i, w_j\right) = \cos\left(\mathbf{v}_i, \mathbf{v}_j\right) = \frac{\mathbf{v}_i \mathbf{v}_j}{\|\mathbf{v}_i\|\|\mathbf{v}_j\|}$$

- 通过直接将词义相似度作为评估目标，可以定量地衡量词向量模型的性能

单词分布式表示的评价与应用

- 常见的内部评估方法：
  - 词义相关性
  - *词语类比性*

- 常见的英文词义相似度评测基准：
  - WordSim-353，包含 353 对英文词语
  - SimLex-999，包含 999 对英文词语
  - 每一对词语都附有一个位于 [0,10] 范围内的相似度分数，如表6-1所示

表 6-1 词义相似度评测基准样本示例

单词 1	单词 2	相似度
dirty	narrow	0.30
student	pupil	9.40
win	dominate	5.68
smart	dumb	0.60
attention	awareness	8.73
leave	enter	1.38

单词分布式表示的评价与应用

- 常见的内部评估方法：
  - 词义相关性
  - *词语类比性*

- 在评估词向量模型时，通常通过计算由词向量得出的词义相似度与标准值之间的 Spearman 或 Pearson 相关系数，来对词向量模型的效果进行评价

- 在应用中，对于给定的单词w，可以通过基于词向量的余弦相似度在词表中检索最相似的词语，即

$$w^* = \arg\max_{w' \in \mathbb{W}} \text{sim}(w, w')$$

单词分布式表示的评价与应用

- 常见的内部评估方法：
  - *词义相关性*
  - 词语类比性

- 在由（man, woman）词对所定义的类比关系下:
  - 通过词向量模型为单词 "son" 检索到类比词 "daughter"
  - 满足“man 对 woman 相当于 son 对 daughter”这一类比关系

单词分布式表示的评价与应用

- 常见的内部评估方法：
  - *词义相关性*
  - 词语类比性

- 通常，对于类似于“w_a之于w_b,相当于w_c之于?”的类比问题，可以通过以下公式来检索最合适的类比词:

$$w^* = \underset{w' \in \mathbb{V}}{\text{argmin}} \cos\left(\mathbf{v}^*, \mathbf{v}_c + \mathbf{v}_b - \mathbf{v}_a\right)$$

  - 其中v^*, v_a, v_b, v_c分别是w^*, w_a, w_b, w_c对应的词向量

主要内容

- 文本的向量表示
- 神经网络语言模型
- 预训练语言模型

链式法则

- 语言模型旨在建模词序列$w_1 w_2 \ldots w_m$的联合概率分布$\mathrm{P}(w_1 w_2 \ldots w_m)$

- 由于联合概率$\mathrm{P}(w_1 w_2 \ldots w_m)$的参数量非常庞大，通常采用链式法则对其进行分解：

$$P(w_1 w_2 \ldots w_m) = P(w_1) P(w_2|w_1) P(w_3|w_1 w_2) \cdots P(w_m|w_1 w_2 \ldots w_{m-1})$$

$$= \prod_{i=1}^{m} P(w_i|w_1 w_2 \cdots w_{i-1})$$

链式法则

- 由此，$w_1 w_2 \ldots w_m$的生成过程可以看作单词逐个生成的过程：
  - 生成w_1
  - 根据w_1生成w_2
  - 根据w_1和w_2生成w_3
  - ……
  - 根据前$m-1$个单词生成最后一个单词w_m
- 通过上述过程将联合概率$\mathrm{P}(w_1 w_2 \ldots w_m)$转换为了多个条件概率的乘积

链式法则

- 例：计算句子 “把努力变成一种习惯” 的概率

$$P(\text{把 努力 变成 一种 习惯}) = P(\text{把}) \times P(\text{努力} \mid \text{把}) \times P(\text{变成} \mid \text{把 努力}) \times$$

$$P(\text{一种} \mid \text{把 努力 变成}) \times P(\text{习惯} \mid \text{把 努力 变成 一种})$$

神经网络语言模型

- 链式法则对联合概率P($w_1 w_2 \ldots w_m$)进行了分解，但模型的参数量仍然没有显著减少
- n元语言模型：一种直接的近似策略
  - 高阶n元语言模型仍然面临着严重的数据稀疏问题
  - 单词的离散表示忽略了单词之间的相似性
- 神经网络语言模型
  - 2000年，Bengio 等人提出了一种基于前馈神经网络估计$P(w_i | w_{i-n+1} \ldots w_{i-1})$的语言模型
  - 循环神经网络、卷积神经网络、端到端记忆网络等神经网络方法
  - 在一定程度上缓解数据稀疏问题，
  - 突破历史信息长度的限制，更好地捕捉长距离的依赖关系
  - 传统的n-gram语言模型已逐渐被神经网络语言模型所取代

神经网络语言模型

神经网络语言模型的技术特点

- 训练过程
  - 采用的是有监督方法
- 训练数据
  - 可以直接从原始文本中获得
  - 仅需要大规模的无标注文本
- 语言模型是典型的自监督学习（Self-supervised Learning）任务

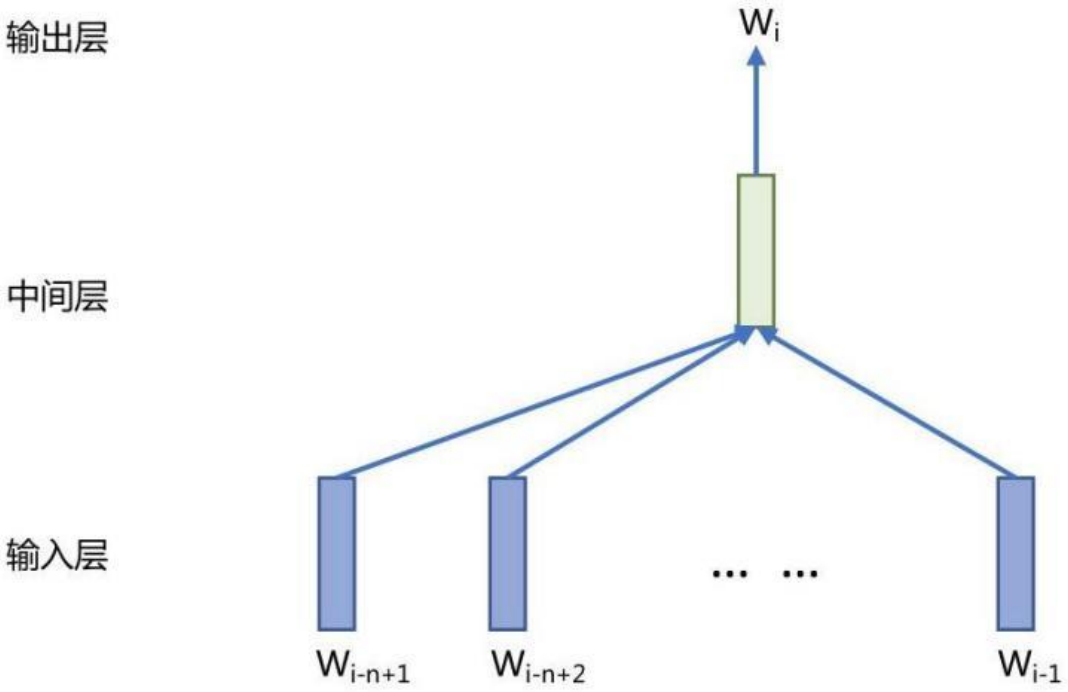
前馈神经网络语言模型

- 给定历史单词序列$w_1 w_2 \ldots w_{i-1}$，神经网络语言模型的目标是根据历史单词对下一时刻词进行预测

- 前馈神经网络语言模型
  - 与传统n元语言模型类似
  - 沿用马尔可夫假设，即下一时刻的词只与过去n-1个词相关
  - 其目标可以表示为输入历史单词$w_{(i-n+1)} \ldots w_{i-1}$，输出词$w_i$在词表$V$上的概率分布，即估计条件概率$P\left(w_i \middle| w_{(i-n+1)}^{i-1}\right)$

前馈神经网络语言模型

- 前馈神经网络由三部分组成，分别为输入层、隐藏层（中间层）和输出层



前馈神经网络语言模型

- 输入层:
  - 将由文本组成的词序列转化为模型可接受的低维稠密向量
  - 具体实现:
    - 根据每个词在词表中的位置，将历史词序列$w_{(i-n+1)},\ldots,w_{i-1}$转换为对应的独热编码（One-Hot Encoding）
    - 将每个词的独热编码映射到一个低维稠密的实数向量
    - 这个映射过程可以看作是通过查找词表获取每个词特有词向量的过程:

$$\mathbf{v} = \left[\mathbf{v}_{(i-n+1)}, \ldots, \mathbf{v}_{i-1}\right]$$

    - 其中，$v_{i-1} \in R^d$代表词w_{i-1}所对应的词向量，d代表词向量的维度，$v \in R^{(n-1)\times d}$代表将所有历史词向量拼接后的结果。

前馈神经网络语言模型

- 隐藏层:
  - 对词向量v进行线性变换与非线性映射
  - 具体计算过程:

$$\mathbf{h} = f\left(\mathbf{W}^{hid}\mathbf{v} + \mathbf{b}^{hid}\right)$$

  - 其中，隐藏层由线性变换矩阵$W^{hid} \in R^{m\times(n-1)d}$、偏置项$b^{hid} \in R^{m}$组成，$m$为隐藏层维度，$f$为非线性激活函数（如Sigmoid、tanh 和 ReLU 等)

前馈神经网络语言模型

- 输出层:
  - 基于隐藏层向量h得到词表空间V上的概率分布
  - 具体计算过程:

$$\mathbf{y} = \text{Softmax}\left(\mathbf{W}^{\text{out}}\mathbf{h} + \mathbf{b}^{\text{out}}\right)$$

  - 其中，$W^{out} \in R^{|V|\times m}$是输出层的线性变换矩阵，$b^{out}$为偏置项，$|V|$为词表大小

前馈神经网络语言模型

- 上述前馈神经网络语言模型的总参数量:
  - $|V| \times d + m \times (n-1)d + m + |V| \times m + |V|$，即$|V|(d + m + 1) + m\big((n-1)d + 1\big)$

- 词向量维度d、隐藏层维度m以及历史词长度$n-1$可根据实际需求进行调整

- 神经网络模型相比传统 n-gram 语言模型的重要优势：
  - 词表大小$|V|$和历史词长度n-1的增大并不会显著增加前馈神经网络语言模型的总参数量，而是保持线性增长的关系

前馈神经网络语言模型

不足

- 在实际应用中，固定长度的历史词并不总能提供足够的信息

- 对于信息较为复杂的长文本，模型可能需要依赖较长的历史才能作出准确预测

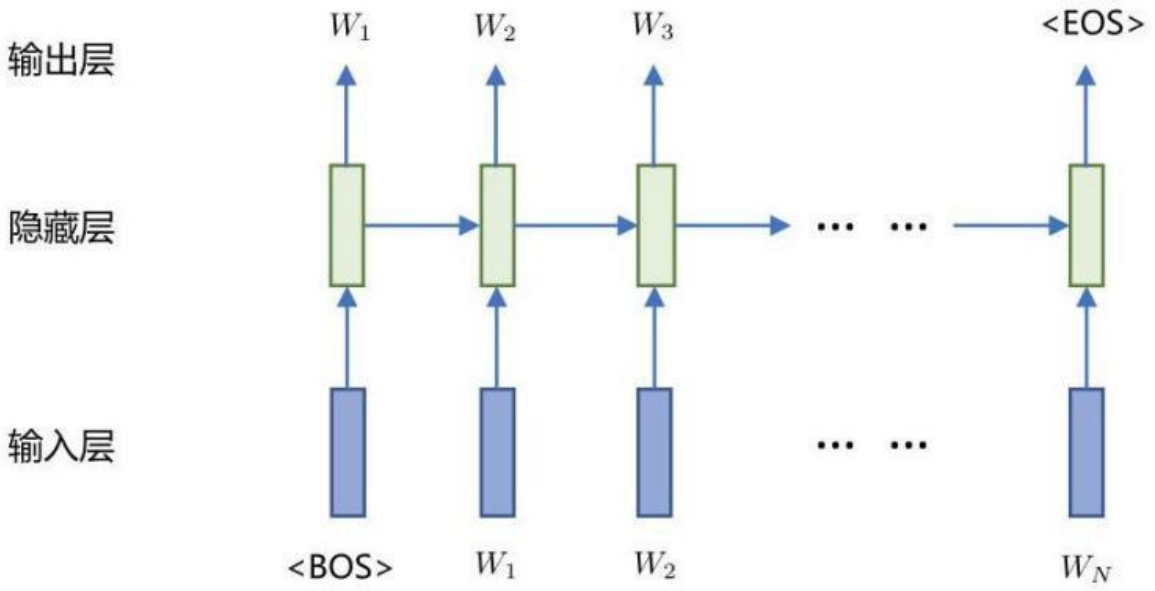
- 例如，“与小明一起旅行游玩总是充满了惊喜，你永远不会知道他将带你到哪里去。”
  - 预测“他”指代的是谁?
  - “小明”和“他”这两个词之间的距离接近 15 个单词，如果仅使用前馈神经网络，所需的历史词长度会过长

循环神经网络语言模型

- 循环神经网络（Recurrent Neural Network, RNN）：
  - 常用于处理序列数据
  - 上一时刻的隐藏层状态会作为当前时刻的输入
  - 每个时刻的隐藏层状态都能够保存并传递过去所有词的信息
  - 通过隐藏层状态动态地捕捉整个序列的依赖关系
  - 有效地解决了语言模型中的动态依赖问题

循环神经网络语言模型

- 循环神经网络（Recurrent Neural Network, RNN）由三部分组成：输入层、隐藏层和输出层



循环神经网络语言模型

- 输入层：
  - 循环神经网络语言模型不限制历史词长度，而是使用整个历史序列
  - 给定历史序列$w_1, \ldots, w_{i-1}$，第i时刻语言模型的目标是预测第i个词w_i
  - 此时循环神经网络语言模型的输入由两部分组成，前一个词w_{i-1}的词向量以及包含所有历史词信息的$i-1$时刻隐藏层输出h_{i-1}

$$\mathbf{x}_i = [\mathbf{v}_{i-1}; \mathbf{h}_{i-1}]$$

循环神经网络语言模型

- 输入层：
  - $x_i \in R^{d+m}$
  - $v_{i-1} \in R^d$代表词w_{i-1}所对应的词向量，d为词向量维度，$h_{i-1} \in R^m$代表前一时刻模型的隐藏层输出，m为隐藏层维度
  - 特别的，对于第 1 个词w_1，由于其没有历史时刻信息，通常使用一个随机初始化向量或0向量h_0作为初始隐藏层向量

$$\mathbf{x}_i = [\mathbf{v}_{i-1}; \mathbf{h}_{i-1}]$$

循环神经网络语言模型

- 隐藏层：
  - 进行线性变化与非线性激活
  - 计算如下：

$$\mathbf{h}_i = f\left(\mathbf{W}^{hid}\mathbf{x}_i + \mathbf{b}^{hid}\right)$$

  - 其中$W^{hid} \in R^{m\times(d+m)}$，$b^{hid} \in R^m$

循环神经网络语言模型

- 隐藏层：
  - 因为x_i可以被分解为两部分，W^{hid}也可以分解为$W^{hid} = [U; V]$，$U \in R^{m \times d}$是词向量w_{i-1}的权重，$V \in R^{m \times m}$是隐藏层输出h_{i-1}的权重
  - 将其拆分开更能体现循环神经网络递归的特点：

$$\mathbf{h}_i = f\left(\mathbf{U}\mathbf{v}_{i-1} + \mathbf{V}\mathbf{h}_{i-1} + \mathbf{b}^{hid}\right)$$

循环神经网络语言模型

- 输出层：
  - 基于隐藏层状态h_i预测词表V上的概率分布
  - 计算如下：

$$\mathbf{y}_i = \text{Softmax}\left(\mathbf{W}^{\text{out}}\mathbf{h}_i + \mathbf{b}^{\text{out}}\right)$$

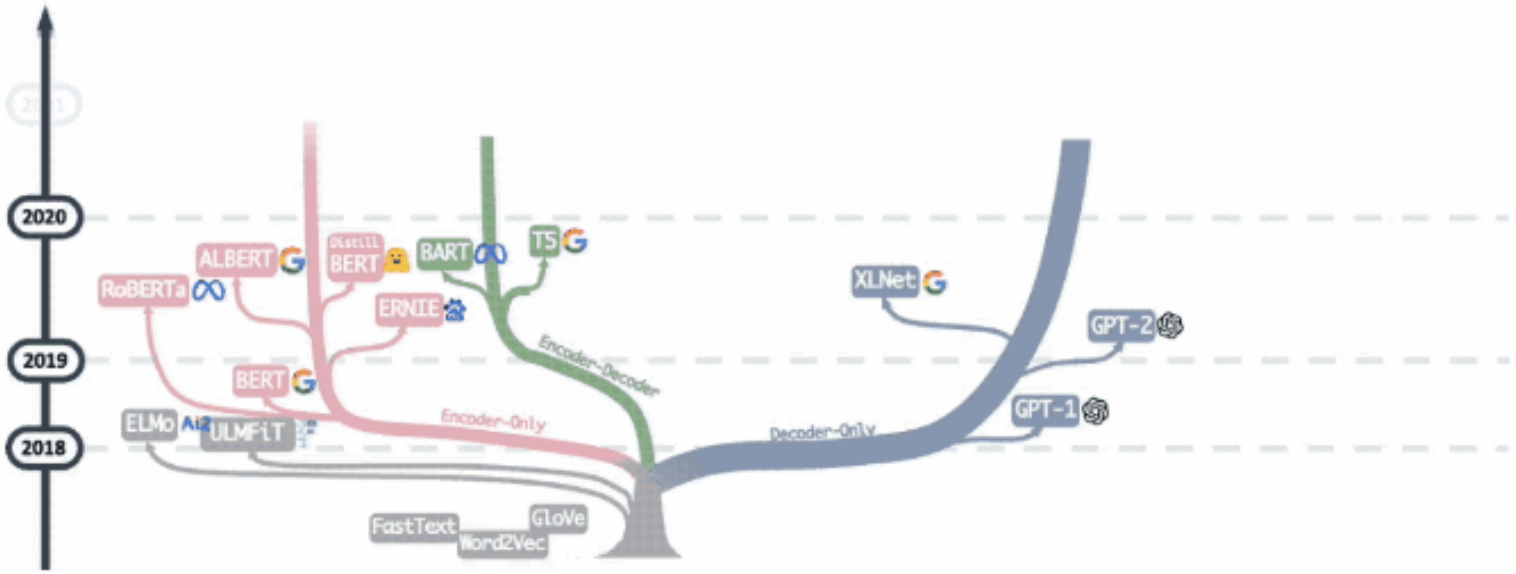
  - 其中，$W^{out} \in R^{|V| \times m}$

循环神经网络语言模型

- 最基础的循环神经网络，其隐藏层结构相对简单
- 在处理长序列时，训练这种基本的循环神经网络可能会遇到梯度消失或梯度爆炸问题，从而导致训练效果不佳
- 解决方法：
  - 在反向传播过程中对梯度进行截断，按序列长度进行限制，但可能会削弱模型捕捉长距离依赖的能力
  - 使用像LSTM这样的具有门控机制的循环神经网络，这类网络往往能实现更加稳定的训练过程，并表现出更好的性能

主要内容

- 文本的向量表示
- 神经网络语言模型
- 预训练语言模型


2020
2019
2018
ALBERT
RoBERTa
DistilBERT
BERT
BART
T5
ERNIE
BERT
ELMo
ULMFiT
Encoder-Only
Encoder-Decoder
Decoder-Only
XLNet
GPT-2
GPT-1
FastText
GloVe
Word2Vec

预训练语言模型

- 随着神经网络语言模型的发展，能够从大规模无标注数据进行自监督学习的预训练语言模型逐渐成为主流

- 自监督学习：预训练 + 微调
  - 通过大量无标注数据进行预训练
  - 通过微调来适应具体任务

- 以ELMo为代表的动态词向量模型是预训练语言模型的早期尝试

- 基于 Transformer 架构的大规模预训练语言模型，如GPT和BERT，标志着这一范式的进一步成熟

预训练语言模型

- 预训练模型
  - 利用大规模语料、自监督任务和深度神经网络结构（如Transformer）学习通用自然语言表示
  - 能够有效捕捉词汇、语法和语义信息
  - 通过微调进行下游任务适配，即使用标注数据对预训练模型进行监督训练，从而实现进一步的性能提升

预训练语言模型

- 本节内容：
  - 以ELMo为代表的动态词向量方法
  - 自回归预训练语言模型GPT
  - 掩码语言模型BERT
  - 序列到序列预练语言模型 BART

动态词向量算法ELMo

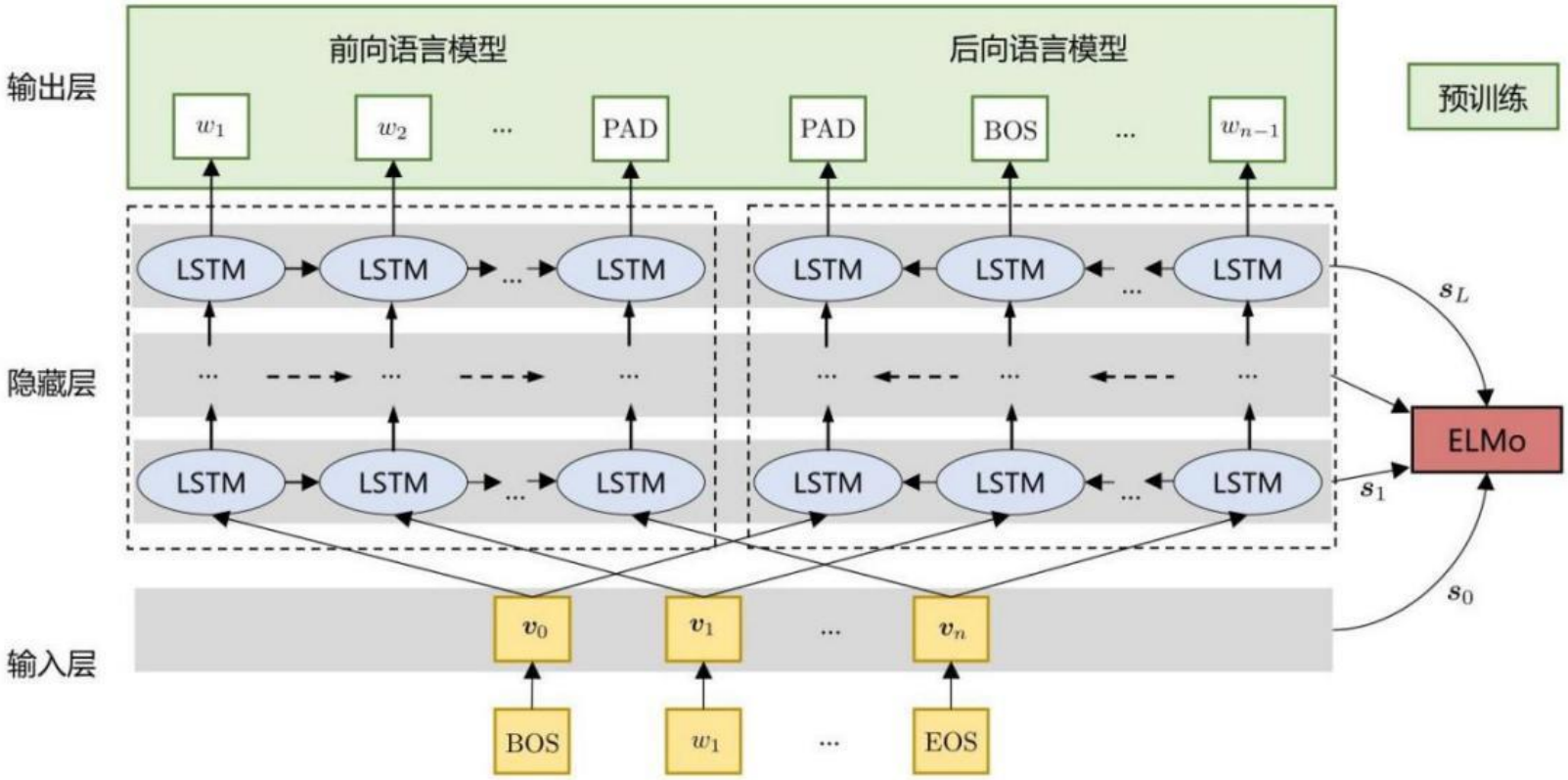
- 根据给定数据集学习得到的词向量是固定的，通常被视为‘静态’的，不随上下文变化
- 自然语言中的词语往往具有多重语义，在不同的上下文或语境中，词语的含义可能会有所不同
- 动态词向量（Dynamic Word Embedding）
  - 也称为上下文相关的词向量（Contextualized Word Embedding）
  - 一个词的向量表示通过其所在上下文动态计算，随着上下文的变化而变化

动态词向量算法ELMo

- ELMo（Embeddings from Language Models）：
  - 深度上下文相关词向量
  - 双向预训练语言模型
    - 从两个方向建模：从左到右进行前向建模和从右到左进行后向建模
    - 双向建模能够提供更丰富的上下文表示，使得文本中的每个词都可以同时利用其左右两侧的信息

动态词向量算法ELMo

- ELMo的神经网络结构如图所示，主要由输入层、隐藏层和输出层三部分组成



动态词向量算法ELMo

- 为了减少词不在词表中 （Out-of-Vocabulary） 的情况，输入层对输入文本进行字符级别的编码

- 具体来说，输入文本中的每个词w_i视作由字符序列$c_{i_1} c_{i_2} \ldots c_{i_m}$组成，每个字符$c_{i_j}$通过字符嵌入层转化为向量$v_{c_{i_j}}$：

$$\mathbf{v}_{c_{i_j}} = \mathbf{W}^{\text{char}} \mathbf{e}_{c_{i_j}}$$

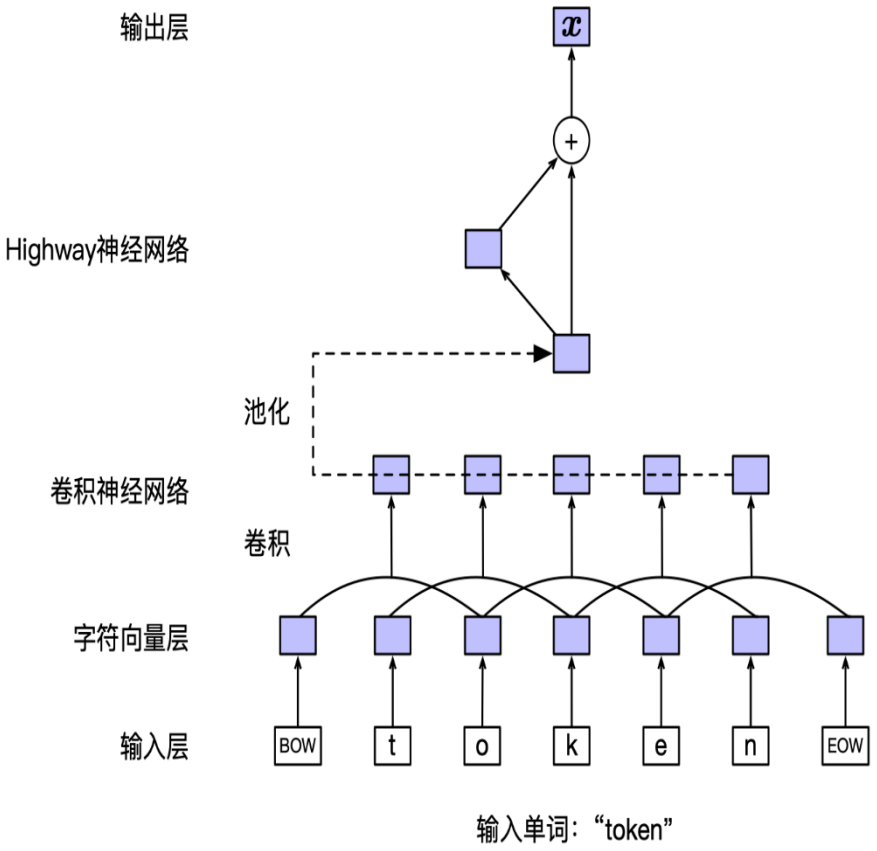
  - 其中，$W^{char} \in R^{d^{char} \times |V^{char}|}$为字符嵌入矩阵、$V^{char}$表示字符库、$d^{char}$表示字符向量维度、$e_{c_{i_j}}$表示字符$c_{i_j}$的独热向量

动态词向量算法ELMo

- 得到词w_i的字符向量表示$v_{c_{i_1}}, v_{c_{i_2}} \ldots, v_{c_{i_m}}$后，ELMo模型使用卷积神经网络对字符级表示进行语义组合

- 调整卷积神经网络的卷积核大小和通道数，可以获取不同粒度的字符级上下文信息

- 在每个位置的卷积输出上应用池化层，从而得到词w_i的词级别表示 $\hat{v}_i$



动态词向量算法ELMo

- 在获得卷积神经网络的输出$\hat{v}_i$后，为了避免梯度消失或爆炸，模型采用 Highway 网络对$\hat{v}_i$进行进一步转换：

$$\mathbf{v}_i = \mathbf{g} \cdot \widehat{\mathbf{v}}_i + (\mathbf{1} - \mathbf{g}) \cdot \mathrm{ReLU}\left(\mathbf{W}\widehat{\mathbf{v}}_i + b\right)$$

  - 其中，g为门控向量，$\hat{v}_i$为卷积神经网络的输出：

$$\mathbf{g} = \sigma\left(\mathbf{W}^g\widehat{\mathbf{v}}_i + \mathbf{b}^g\right)$$

  - 其中，W^g为线性变换矩阵、b^g为偏置项

动态词向量算法ELMo

- 在获得每个词的上下文无关词向量后，ELMo 的编码层将从两个方向进一步对词向量进行编码
- ELMo使用了两个独立的编码器，分别对前向和后向进行语言模型建模
- 在预训练过程中，分别使用正向和反向 LSTM 的最高层输出$\overrightarrow{h}_{i,L}$与$\overleftarrow{h}_{i,L}$来预测下一时刻的词
- 对于给定的文本序列$w_1 w_2 \ldots w_n$，前向语言模型的目标词是在t时刻的目标词w_{t+1}，而后向语言模型的目标词则是w_{t-1}

动态词向量算法ELMo

- 采用前向和后向语言模型的建模过程可以表示为：

$$P_{\text{forward}}\left(w_1 w_2 \ldots w_n\right) = \prod_{i=1}^{n} P\left(w_i | w_{1:i-1}; \boldsymbol{\theta}_f\right)$$

$$P_{\text{backward}}\left(w_1 w_2 \ldots w_n\right) = \prod_{i=1}^{n} P\left(w_i | w_{i+1:n}; \boldsymbol{\theta}_b\right)$$

  - 其中，θ_f和θ_b分别指代表前向和后向 LSTM 模型的参数
  - 双向模型共享输出层的参数

动态词向量算法ELMo

- ELMo算法的编码层采用了多层双向 LSTM 结构
- 通常认为，模型的低层能够捕捉语法等基础语言特征，而高层则能够捕捉语义和语境等更深层次的语言特征
- 双向 LSTM 的使用确保了在编码过程中，每个位置的词都能同时获得该位置过去和未来词的上下文信息

动态词向量算法ELMo

- 对于词w_i来说，一个L层的 ELMo 模型会产生$2L+1$个向量表示：

$$R_i = \left\{ \mathbf{v}_i, \overrightarrow{\mathbf{h}}_{i,j}, \overleftarrow{\mathbf{h}}_{i,j} | j = 1, \ldots, L \right\}$$

  - v_i代表输入层得到的上下文无关词向量
  - $\vec{h}_{i,j}$代表第j层前向 LSTM 编码得到的特征
  - $\overleftarrow{h}_{i,j}$代表第j层后向 LSTM 编码得到的特征

- 对于每层得到的两个方向的特征， ELMo 将其拼接起来得到$h_{i,j}^{LM} = [\vec{h}_{i,j}; \overleftarrow{h}_{i,j}]$

动态词向量算法ELMo

- 在进行下游任务时，ELMo会将R_i中的所有向量整合成一个向量
  - 最简单的情况：直接使用最后一层的表示$h_{i,j}^{LM}$

- 每一层 LSTM 学到的信息不同，对于不同的任务，每层特征的重要性也有所不同
  - 常见的做法：根据任务所需的信息，对每一层的特征进行加权，从而得到词w_i对应的 ELMo 向量

$$\text{EMLo}_i^{\text{task}} = \gamma^{\text{task}} \sum_{j=0}^{L} s_j^{\text{task}} \mathbf{h}_{i,j}^{LM}$$

动态词向量算法ELMo

- 其中，γ^{task}是整体的缩放系数，s^{task}是每层的权重系数，反映每一层向量对于目标任务的重要性
- 在执行下游任务时，一般将v_i和$ELMo_i^{task}$拼接起来作为词w_i的最终表示向量
- 可以使用γ^{task}对 ELMo 向量进行适当缩放
- s^{task}通常在下游任务的训练过程中学习得到
- 而 ELMo 模型的编码器参数在下游任务训练时则被 “冻结”，不参与更新

- ELMo相比GloVe（静态词向量）在词义消歧和近邻分析任务上都有比较好的表现

模型	词	近邻
GloVe	play	playing, game, games, played, players, plays, player, Play, football, multiplayer
ELMo	Chico Ruiz made a spectacular play on Alusik's grounder · · ·	Kieffer , the only junior in the group , was commended for his ability to hit in the clutch , as well as his all-round excellent play
	Olivia De Havilland signed to do a Broadway play for Garson · · ·	· · · they were actors who had been handed fat roles in a successful play , and had talent enough to fill the roles competently , with nice understatement

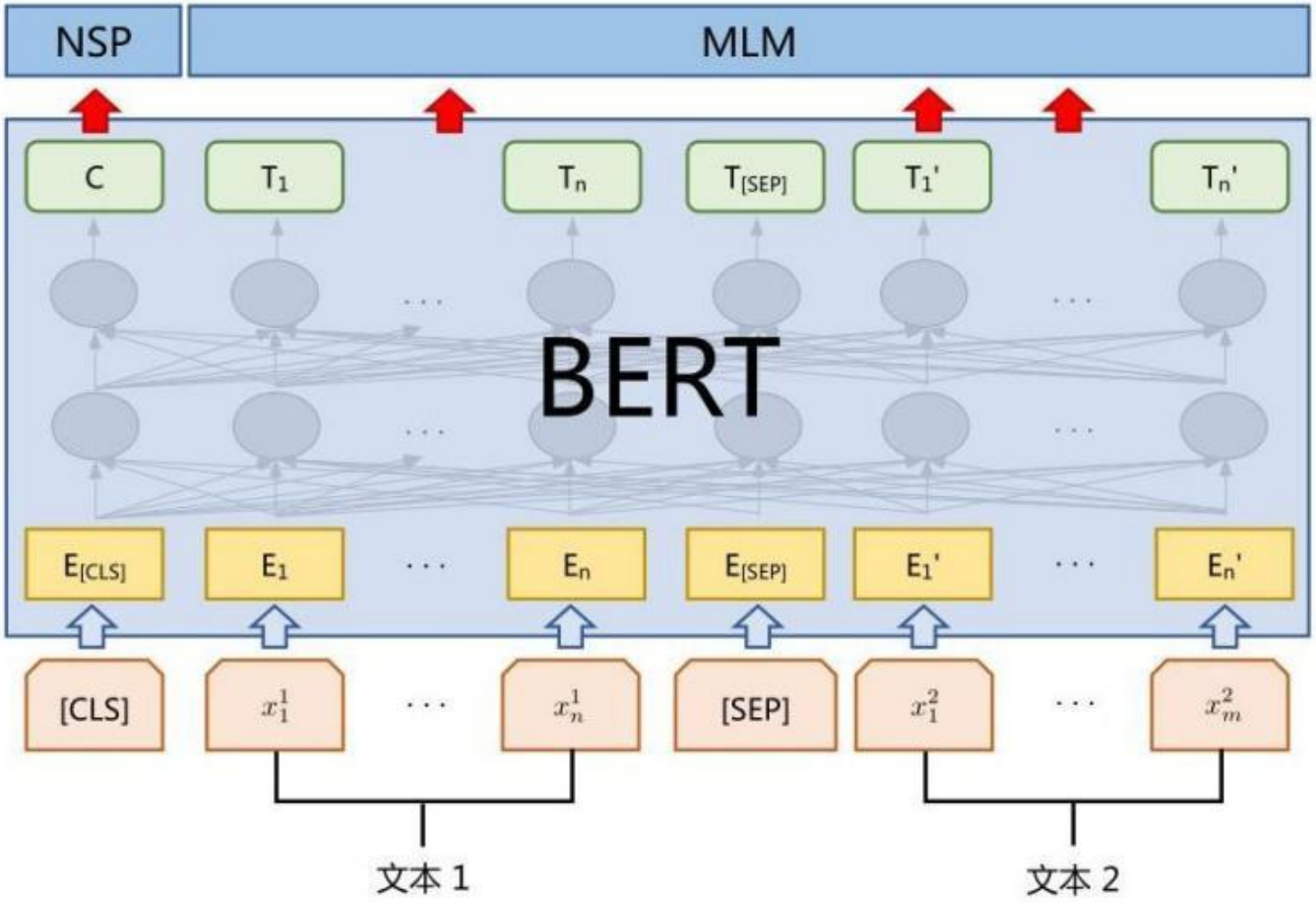
掩码预训练语言模型BERT

- 2018年，Devlin等人提出了掩码预训练语言模型BERT（Bidirectional Encoder Representations from Transformers），是一个基于Transformer架构的双向预训练语言模型

- BERT：
  - 通过双向上下文建模捕捉文本的全局语义信息
  - 更适合理解类任务，如文本分类和问答
  - 利用自注意力机制实现更高效的动态上下文交互
  - 创新性地提出了掩码语言建模（Masked Language Model, MLM）
  - 但生成能力较弱，计算资源需求较高

掩码预训练语言模型BERT

- 如图，BERT的模型分为输入层、编码层和输出层



掩码预训练语言模型BERT

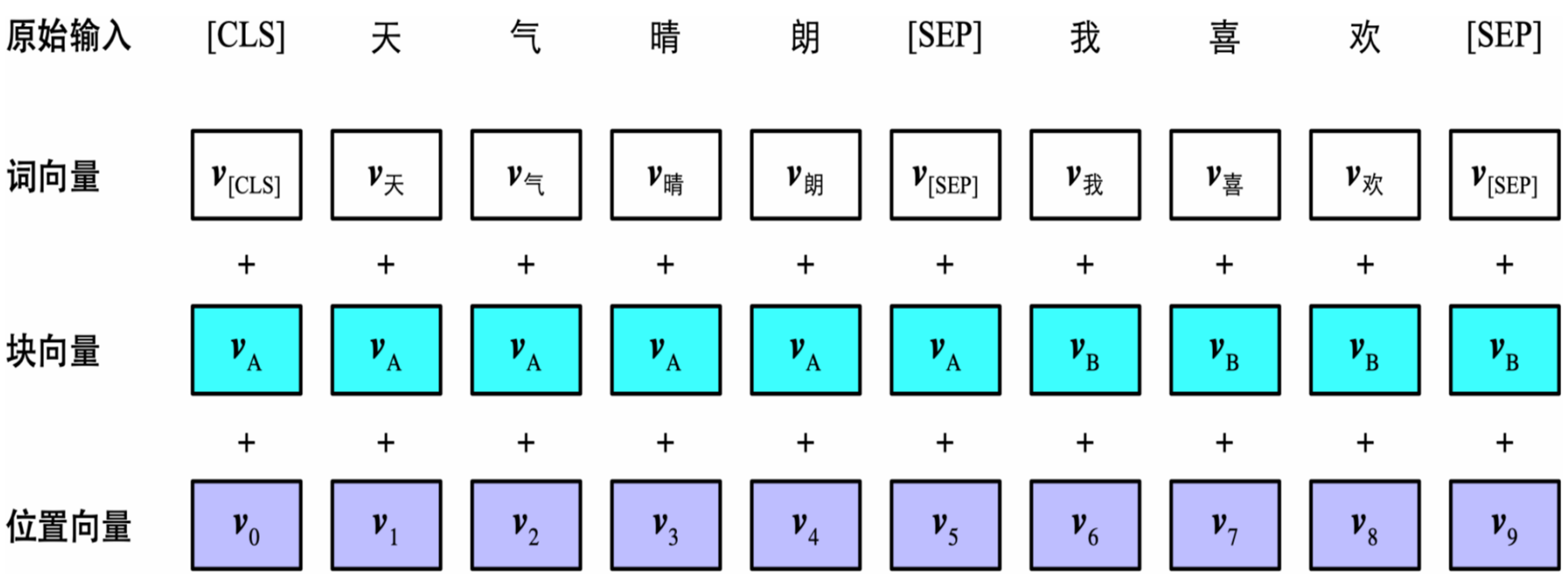
- BERT 的输入层采用了 WordPiece 分词模型：
  - 一种子词表示方法，与 BPE主要区别有2点
  - 区别1：添加前缀： highest→ high/ ##est/ 两个子词，“##” 表示这是一个子词且非单词起始位置
  - 区别2：采用互信息I(X,Y)来衡量两个子词是否合并
    - 当I(x,y)>0 时，说明子词组合(x,y) 的出现并非随机，而是存在语言学意义上的关联。
  - 这种统计语言学方法能有效捕捉：
    - 词根与词缀的固定组合（如＂play＂＋＂##ing＂）
    - 复合词的组成规律（如＂basket＂＋＂ball＂）
    - 高频短语的粘合特征（如＂New＂＋＂York＂）

掩码预训练语言模型BERT

- 在对输入文本进行分词后，BERT 的输入表示由三部分嵌入向量组成，分别为：
  - 词嵌入（Token Embedding）：
    - 表示分词后的每个子词，通过查表从预训练词汇表中获取其对应的向量
  - 段嵌入（Segment Embedding）：
    - 用于区分输入文本中的两段拼接句子
    - 通常，句子 A 的段嵌入表示为全零向量，句子 B 的段嵌入表示为全一向量
    - 这种设计支持下一句预测（NSP）任务的建模
  - 位置嵌入（Position Embedding）：
    - 用于保留输入序列中词汇的位置信息
    - Transformer 本身不显式建模序列顺序，位置嵌入帮助模型感知词汇的排列次序

掩码预训练语言模型BERT

- 三部分嵌入向量被逐元素相加，形成最终的输入表示



掩码预训练语言模型BERT

- 形式化地，每个词的输入表示v可以表示为：

$$\mathbf{v} = \mathbf{v}^t + \mathbf{v}^s + \mathbf{v}^p$$

  - v^t代表词嵌入
  - v^s代表段嵌入
  - v^p代表位置嵌入
  - 三种嵌入维度需要相同，这里设为e
- BERT 的输入层
  - 既能灵活处理长文本中的细粒度语义信息
  - 又能适应复杂的上下文关系
  - 为模型的强大性能奠定了基础

掩码预训练语言模型BERT

- BERT 的编码层
  - 由多个Transformer编码器层堆叠而成
  - 使用L表示所采用的层数，H表示每层的隐藏单元数(维度)，A是指自注意力头数量
  - 原文提供了两种规模的预训练模型：
    - $BERT_{BASE}$使用$L=12$，$H=768$，$A=12$总参数量为$110M$
    - $BERT_{LARGE}$使用$L=24$，$H=1024$，$A=16$总参数量为$340M$
- BERT采用了双向多头自注意力机制（Bidirectional Multi-Head Self-Attention）
  - 不仅关注左侧的上下文，还同时关注右侧的上下文
  - 大幅增强其在语义理解任务中的性能
  - 不适用于生成任务场景

掩码预训练语言模型BERT

- 预训练任务：
  - 掩码语言建模（MLM）
    - 随机掩盖部分词汇，并让模型预测这些词
    - 学习双向上下文
  - 下一句预测（NSP）
    - 让模型判断两段文本是否连续出现
    - 帮助理解句间关系

掩码预训练语言模型BERT

- 掩码语言建模：
  - BERT的核心预训练任务之一
  - 类似完形填空的任务
  - 随机掩盖输入文本中 15% 的子词，让模型根据剩余上下文预测被掩盖的词
  - 同时编码左右两侧的上下文信息，生成更加全面的词表示
  - 并非总是简单地用特殊标记 [MASK] 替换被掩盖的词，因为 [MASK] 在下游任务中不会出现，这可能导致预训练与下游任务之间的分布不一致

掩码预训练语言模型BERT

- 掩码语言建模：
  - 三种掩盖操作：
    - 80% 的概率将词替换为 [MASK]，要求模型直接预测掩盖词
    - 10% 的概率随机替换为词表中的任意词，增加模型的鲁棒性
    - 10% 的概率保持原词不变，鼓励模型在所有位置学习上下文依赖
  - 更精准地预测上下文语义
  - 提高了模型泛化能力
  - 实现了上下文语义的双向建模
  - 提升了对词和上下文的语义理解

掩码预训练语言模型BERT

- 下一句预测：
  - BERT模型的第二个预训练任务
  - 旨在增强模型理解文本间关系的能力

- 尽管通过掩码语言建模，BERT 可以高效构建单段文本的语义表示，但对于需要处理两段文本的任务（如阅读理解和自然语言推断），仅依赖单段建模是远远不够的

- 引入NSP任务，以学习两段文本之间的语义关联

- 任务目标：判断两段文本是否具有上下句的关系

掩码预训练语言模型BERT

- 下一句预测：
  - 从语料中随机选取句子对进行训练：
    - 正样本：句子 A 和句子 B 是语料中相邻的连续句子，构成上下句关系
    - 负样本：句子 B 从其他段落随机选取，与句子 A 无直接语义关联
    - 正负样本的比例为 1:1
  - BERT 学习到句子间的语义联系和逻辑顺序，使其在句对关系建模任务中表现出色
  - 拓展了 BERT 的适用范围
    - 能处理单段文本语义
    - 能高效建模句间关系

补充材料：深入理解BERT



BERT模型有哪些？

- 最常见的就是BERT-base和BERT-large [H是词向量维度，A是注意力的head数]
  - base：L=12,H=768,A=12,Total Prameters=110M
  - large：L=24,H=1024,A=16,Total Prameters=340M
- 但是实际上谷歌开源出来一堆BERT

	H=128	H=256	H=512	H=768
L=2	2/128 (BERT-Tiny)	2/256	2/512	2/768
L=4	4/128	4/256 (BERT-Mini)	4/512 (BERT-Small)	4/768
L=6	6/128	6/256	6/512	6/768
L=8	8/128	8/256	8/512 (BERT-Medium)	8/768
L=10	10/128	10/256	10/512	10/768
L=12	12/128	12/256	12/512	12/768 (BERT-Base)

补充材料：深入理解BERT



BERT模型有哪些？

- 一般来说，越大的BERT，效果越好

Model	Score	CoLA	SST-2	MRPC	STS-B	QQP	MNLI-m	MNLI-mm	QNLI(v2)
BERT-Tiny	64.2	0.0	83.2	81.1/71.1	74.3/73.6	62.2/83.4	70.2	70.3	81.5
BERT-Mini	65.8	0.0	85.9	81.1/71.8	75.4/73.3	66.4/86.2	74.8	74.3	84.1
BERT-Small	71.2	27.8	89.7	83.4/76.2	78.8/77.0	68.1/87.0	77.6	77.0	86.4
BERT-Medium	73.5	38.0	89.6	86.6/81.6	80.4/78.4	69.6/87.9	80.0	79.1	87.7

- 但是太大的预训练模型，当时一般的显卡调不动
  - 不是预训练训不动，是微调都调不动，单张显卡连模型都装不下

补充材料：深入理解BERT



- **可解释性**
  - 自解释：在模型构建之初针对性地设计其结构，使其具备可解释性
  - 事后解释：对于 BERT 等大规模预训练模型的解释性研究，集中在此类
- **两个角度**
  - 自注意力机制
  - 表示学习
- **两种方法**
  - 可视化（Visualization）
  - 探针实验（Probing）

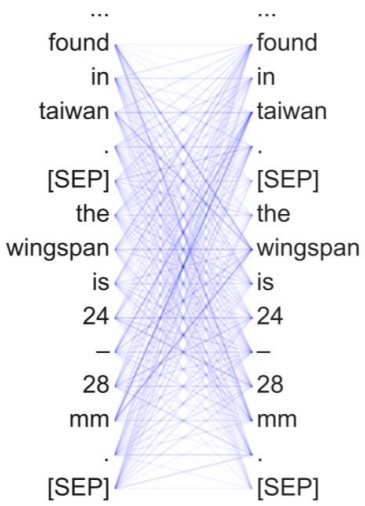
补充材料：深入理解BERT



- **自注意力可视化分析**
  - 自注意力的分析将有助于理解 BERT 模型对于关系（relational） 特征的学习能力

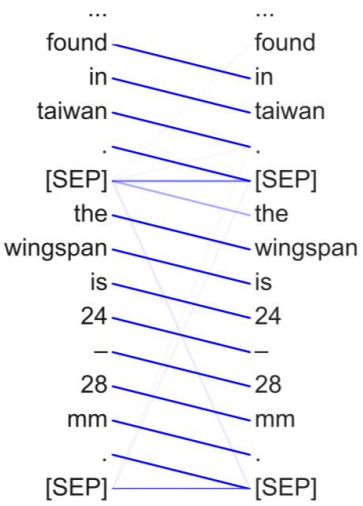
自注意力头1–1

注意力较分散



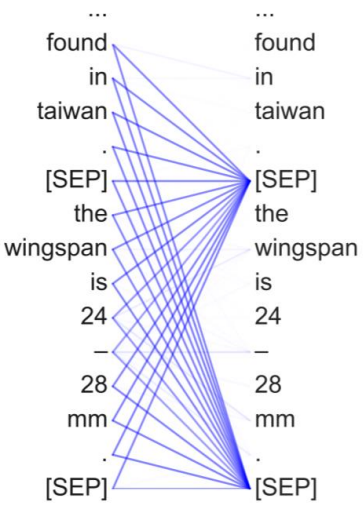
自注意力头3–1

集中在下一个单词



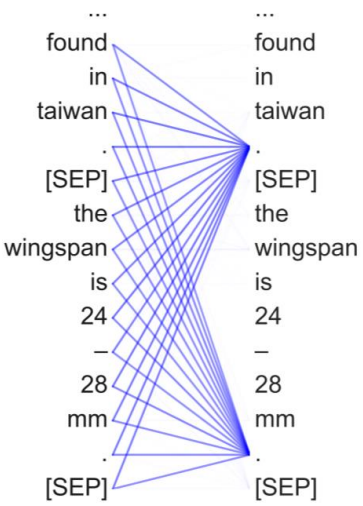
自注意力头8–7

集中在[SEP]标记



自注意力头11–6

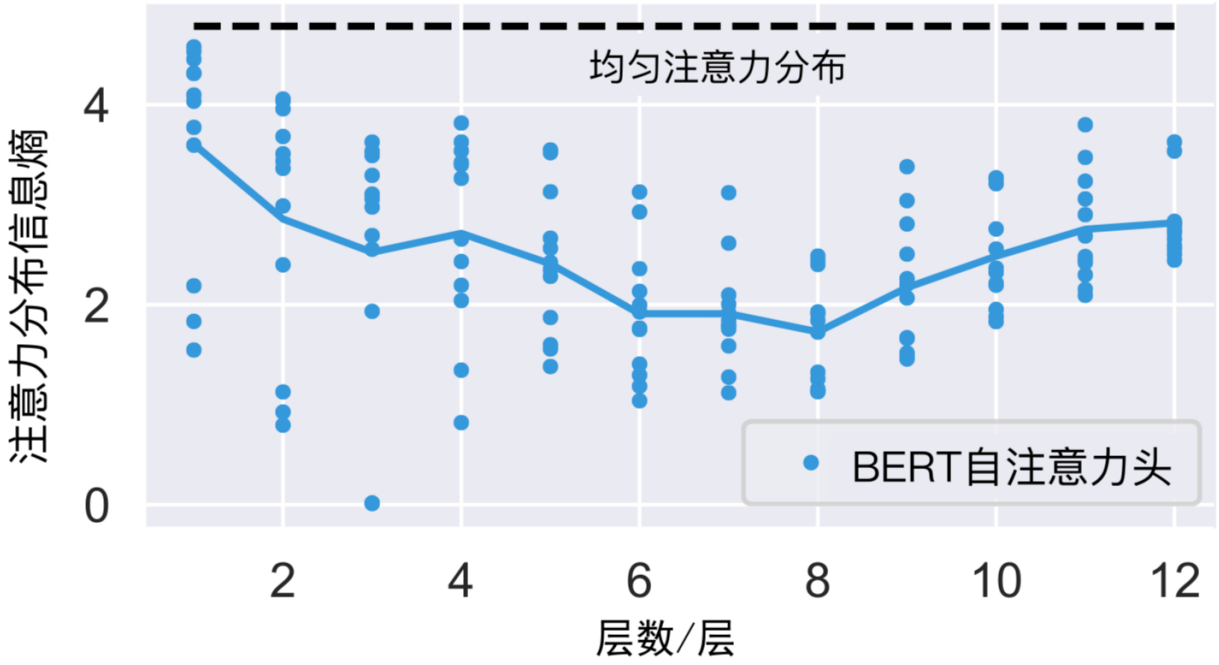
集中在句号



补充材料：深入理解BERT



- **自注意力可视化分析**
  - 计算各层注意力分布的信息熵
  - 浅层阶段熵值较大，中间阶段熵值减小，深层阶段熵值再次增大
  - 一定程度上可以反映 BERT模型中信息聚合（或语义组合）的过程



补充材料：深入理解BERT

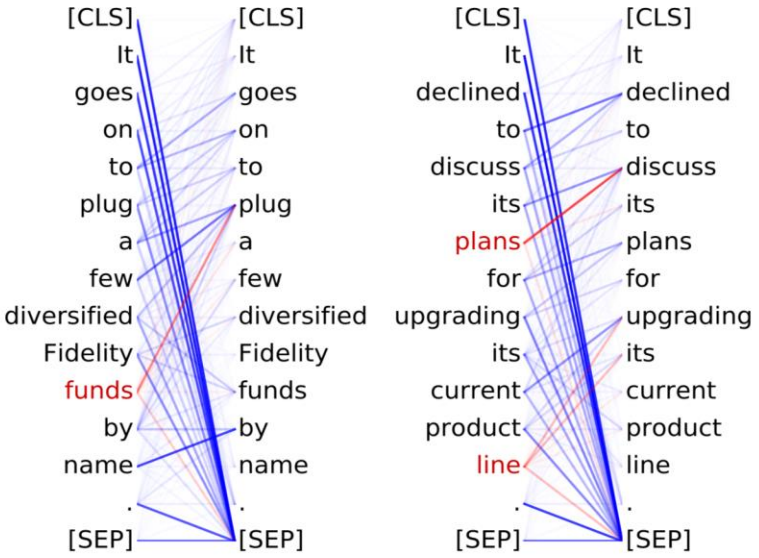


探针实验

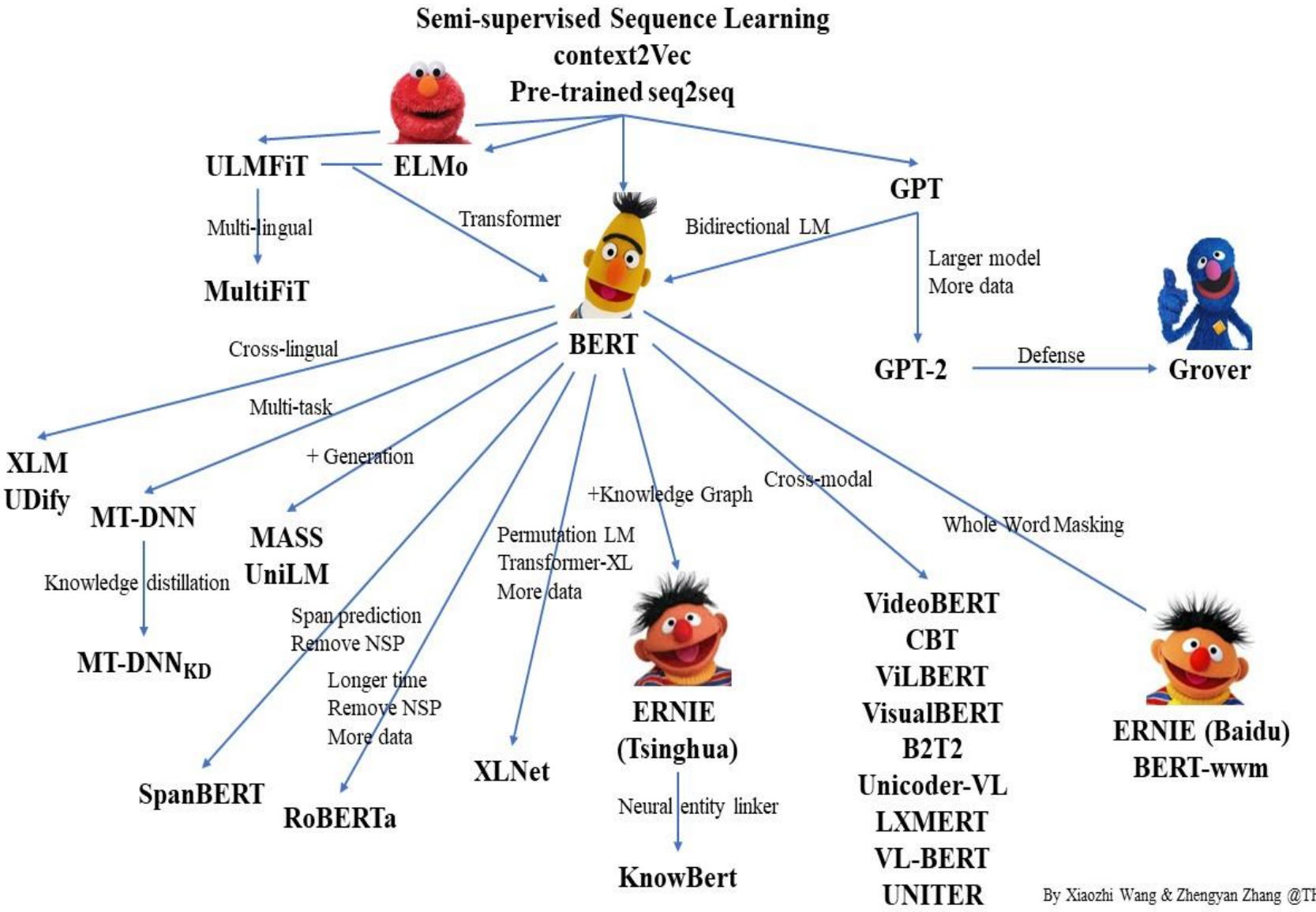
- 设计特定的探针，对于待分析对象（如自注意力或隐含层表示）进行特定行为分析
- 探针通常是一个非参或者非常轻量 的参数模型（如线性分类器），它接受待分析对象作为输入，并对特定行为预测
- 预测的准确度可以作为待分析对象是否具有该行为的衡量指标

例： 为了检验某个自注意力头对直接宾语（Direct object，dobj）关系的表达能力，可以设计一个探针对该自注意力头在dobj句法关系预测上的表现进行分析

- 在BERT第8层第10个自注意力头（记为8-10号）的注意力分布中，其中红色高亮部分即为dobj关系
- 结果表明在BERT模型中，确实存在一部分自注意力头较好地捕捉到特定的句法关系



Bert开启了预训练模型时代



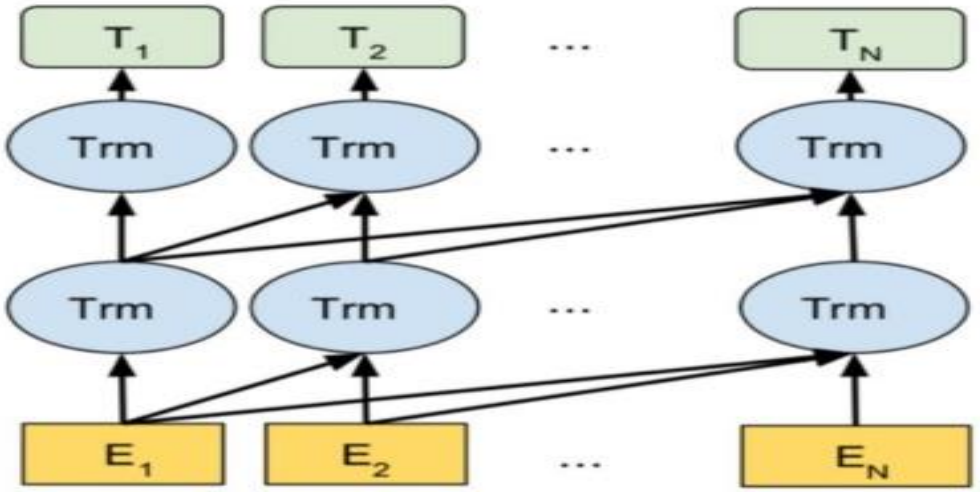
自回归预训练语言模型GPT

- OpenAI于2018年提出的 GPT（Generative Pre-Training）模型，是生成式预训练语言模型的典型代表之一

- GPT：
  - 单向自回归，即模型仅依赖于当前及之前的词汇来预测下一个词，避免了双向上下文的信息交互
  - 训练过程分为两大阶段：
    - 生成式预训练阶段：通过对大规模文本进行自监督学习，学习到语言的基本结构和规律
    - 判别式任务的精调阶段：通过微调（fine-tuning）来适应具体的下游任务，如文本分类、情感分析等判别式任务

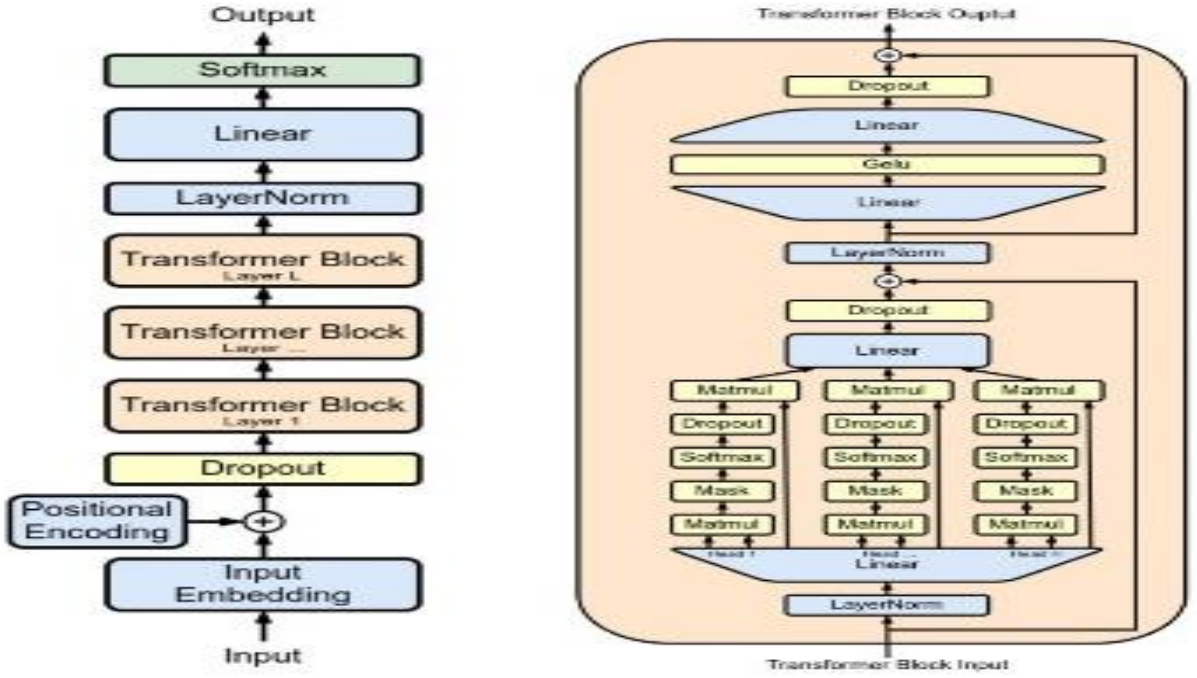
自回归预训练语言模型GPT

- GPT模型的“单向”特性意味着模型在处理文本序列时，只能从左到右或从右到左进行建模，如图所示



自回归预训练语言模型GPT

- GPT模型的结构如图所示，通常包括输入层、编码层和输出层三大部分
  - 其中编码层由多层 Transformer 构成，且是单向的语言模型



自回归预训练语言模型GPT

- 单向建模的方式：
  - 模型在训练过程中遵循严格的因果关系，即每个词的生成只能依赖于它之前的词汇信息，而不能利用未来的上下文

- 具体来说，GPT 使用的 Transformer 架构及其解码策略确保了输入文本序列中的每个位置仅能够依赖于先前时刻的信息
  - 与ELMo模型形成了鲜明对比
  - 在生成式任务中，因果建模方式尤其重要

自回归预训练语言模型GPT

- 自监督预训练使得GPT模型具备了一定的通用语义表示能力
- 下游任务精调（Fine-tuning）的目的是在通用语义基础上对下游任务进行适配
- 下游任务精调需要利用标注数据进行训练
  - 数据集使用D进行表示
  - 每个样例由输入长度为n的文本序列$x = x_1 x_2 \ldots x_n$和对应的标签y构成

自回归预训练语言模型GPT

- 精调过程：
  - 将文本序列x输入GPT模型，获得最后一层对应最后一个词的隐藏层输出$h_n^{(L)}$
  - 通过全连接层进行变换，再结合Softmax 函数，最终得到标签预测结果

$$P\left(y|x_1 \ldots x_n\right) = \text{Softmax}\left(\mathbf{h}^{(L)}\mathbf{W}^y\right)$$

  - 其中，$W^y \in R^{d\times k}$为全连接层参数，k为标签个数

自回归预训练语言模型GPT

- 通过对整个标注数据集D进行优化，精调下游任务的目标函数：

$$\mathcal{L}^{\mathrm{FT}}(\mathbb{D}) = \sum_{(x,y)} \log P\left(y|x_1 \ldots x_n\right)$$

自回归预训练语言模型GPT

- 灾难性遗忘（Catastrophic Forgetting）：
  - 模型在精调后遗忘预训练阶段所学习到的通用语义知识表示
  - 损失模型的通用性和泛化能力

- 为了缓解这一问题，通常采用混合预训练任务损失和下游精调损失的方式：

$$\mathcal{L} = \mathcal{L}^{\mathrm{FT}}(\mathbb{D}) + \lambda \mathcal{L}^{\mathrm{PT}}(\mathbb{D})$$

  - 其中λ取值为$[0,1]$，用于调节预训练任务损失占比

GPT和BERT对比

- GPT
  - 适合自然语言生成
  - 自回归式单向预训练语言模型
  - 由于缺乏双向上下文语境信息，在捕获输入语义整体性方面有所不足

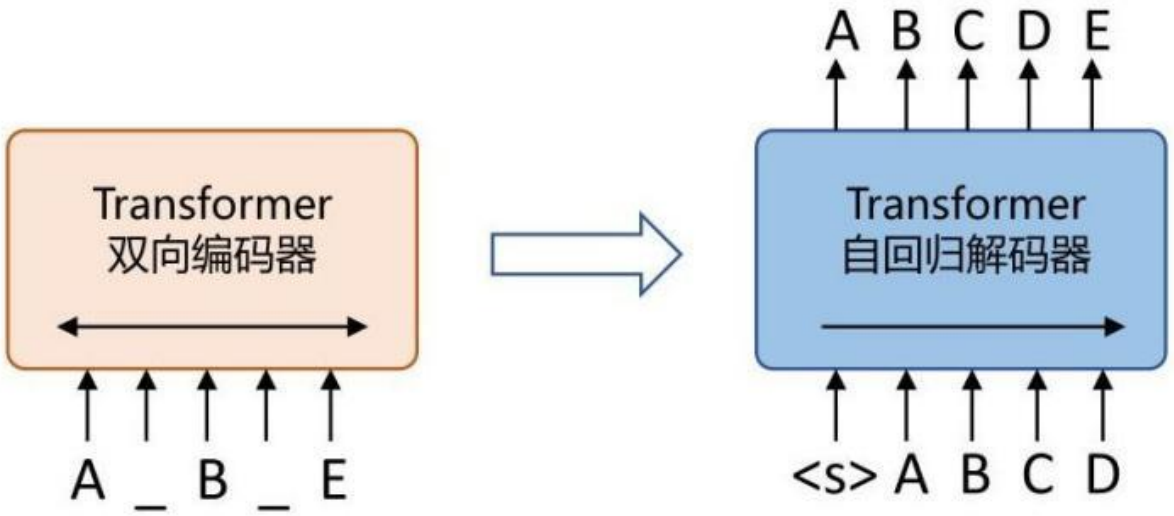
- BERT
  - 适合自然语言理解任务
  - 掩码预训练语言模型
  - 利用掩码语言模型进行预训练，能够同时考虑输入文本的前后文信息
  - 然而生成能力较弱，其掩码机制和预训练目标限制了其在自然语言生成任务中的表现

序列到序列预训练语言模型BART

- BART（Bidirectional and Auto-Regressive Transformers）：
  - 同时满足自然语言生成和理解需求
  - 结合 BERT 和 GPT 的优势
  - 编码器采用双向 Transformer，可以充分利用上下文信息
  - 解码器则采用自回归方式，能够高效地生成自然语言
  - 成为自然语言生成任务中的强大工具，同时也能处理复杂的理解任务

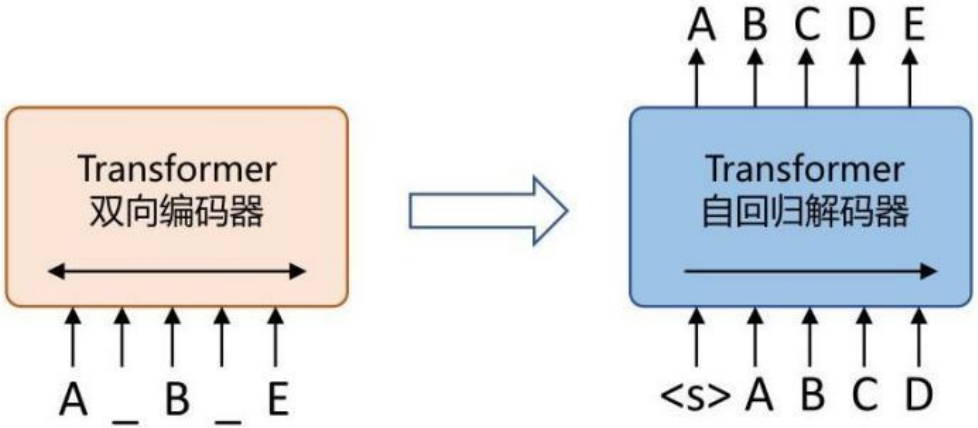
序列到序列预训练语言模型BART

- BART模型的基本结构：
  - 结合双向 Transformer 编码器以及单向的自回归解码器
  - 序列到序列结构



序列到序列预训练语言模型BART

- BART模型采用的优化设计：
  - 使用 GeLU（Gaussian Error Linear Unit）作为激活函数，相较于 ReLU（Rectified Linear Unit）能更好地平滑梯度，提高模型的训练效率和性能
  - 参数初始化采用了正态分布 N(0,0.02)，在训练初期提供更稳定的收敛特性



序列到序列预训练语言模型BART

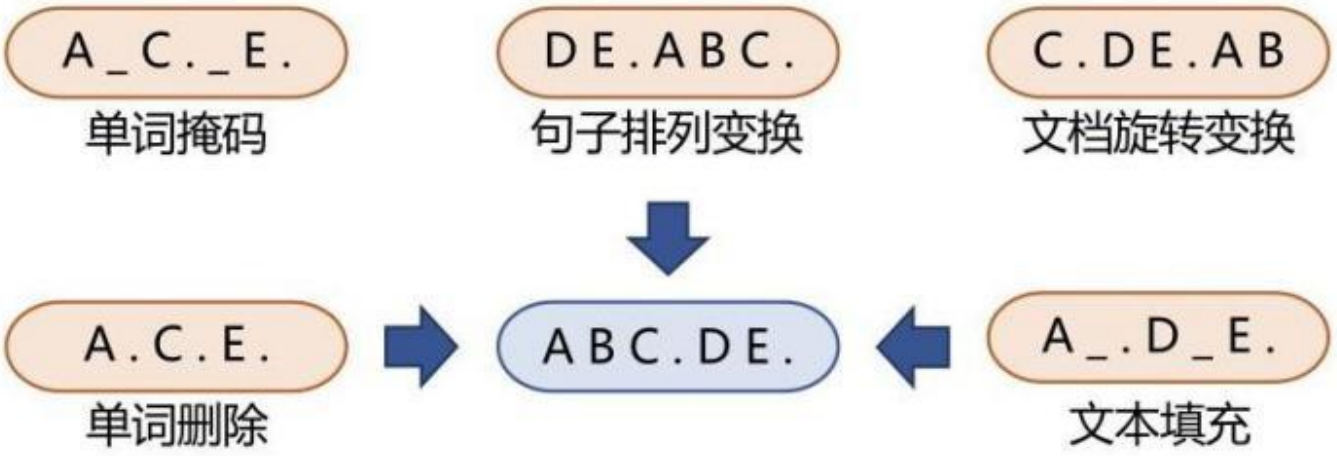
- BART的创新不仅体现在结构设计上，还包括其预训练任务的选择

- BART的预训练任务：
  - 包括多种噪声添加和去噪任务
    - 例如文本随机删除、句子顺序重排和文本填充（Text Infilling）
  - 模拟真实场景中的噪声问题
  - 提升模型在生成任务中的泛化能力

- 通过在大规模数据上预训练，BART 能够生成更连贯、上下文相关性更强的自然语言，同时在某些理解任务中也展现出强大的性能

序列到序列预训练语言模型BART

- 五种加噪方式：
  - 全面模拟不同的自然语言处理场景
  - 可以对这些方式进行组合使用



序列到序列预训练语言模型BART

- 五种加噪方式：
  - 单词掩码（Token Masking）：
    - 随机选择部分单词替换为掩码标记（[MASK]），类似于 BERT，训练模型预测被掩盖的单词
  - 单词删除（Token Deletion）：
    - 随机删除部分单词，要求模型不仅能够预测缺失的单词，还需推断其应在的位置
  - 文本填充（Text Infilling）：
    - 将多处连续单词（文本片段）替换为单个掩码标记，片段长度服从泊松分布（λ=3）
    - 需要模型判断片段长度并重建缺失内容
  - 句子排列变换（Sentence Permutation）：
    - 将文本分割成多个句子后随机打乱顺序，训练模型理解句间关系并复原正确顺序
  - 文档旋转变换（Document Rotation）：
    - 随机选择一个单词作为新文档开头，旋转文本顺序，要求模型找出原始文本的起始位置

序列到序列预训练语言模型BART

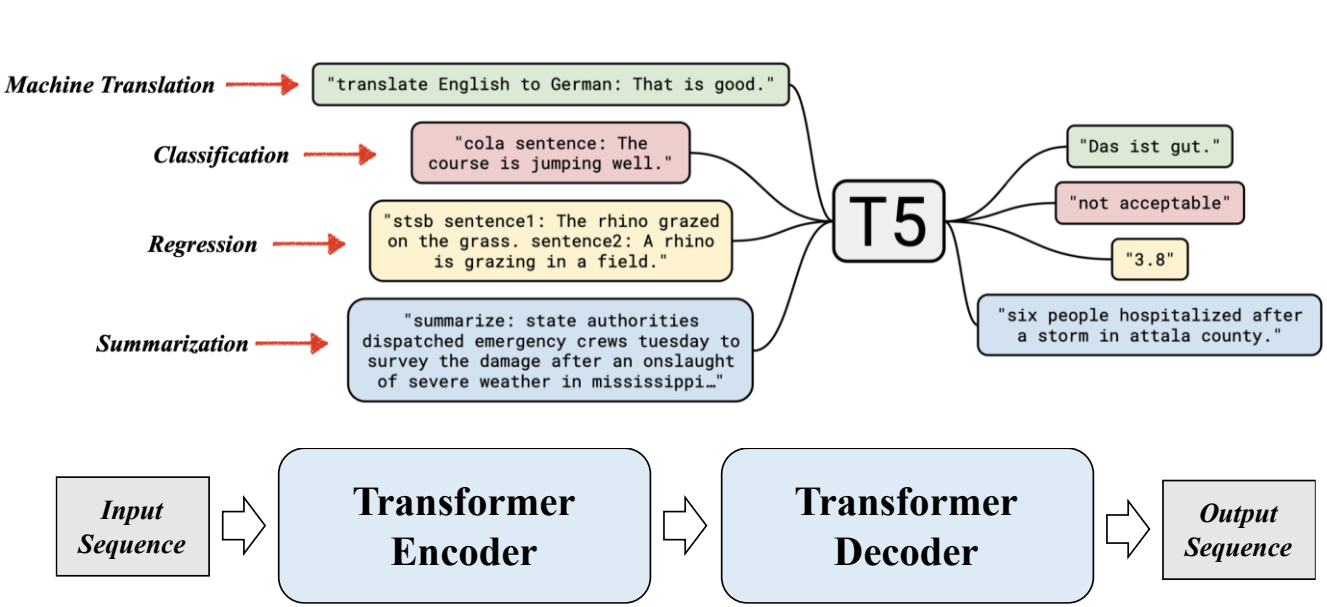
- 多层次的噪声机制涵盖了单词、句子和文档级别的重构任务，适配各种生成场景
- 实验表明，文本填充任务在下游任务中表现突出，结合句子级别的去噪任务还能进一步提升性能
- 尽管 BART 的预训练任务主要面向自然语言生成，它在许多理解任务上也表现优异

序列到序列预训练语言模型BART

- BART模型精调
- 文本理解任务：如序列分类
  - BART 的输入同时传递给编码器和解码器，但模型的语义表示主要来自解码器的最终时刻隐藏层状态
  - 解码器最后的隐藏状态被用作输入文本的语义向量表示，并通过线性分类器预测对应的标签
  - 在句子末尾添加特殊标记，通过该标记位置的隐藏状态表示整体语义，再结合标注数据调整模型参数
- 文本生成任务：如生成式文本摘要（Abstractive Summarization）或生成式问答（Abstractive Question Answering）
  - 精调时输入为任务定义的文本，目标是通过解码器生成符合任务要求的输出文本
  - 生成的文本与任务目标文本之间的差异构成模型优化的损失函数

补充：序列到序列预训练语言模型T5

- **T5: Text-to-Text Transfer Transformer**
  - 提出一种适用于各种类型NLP任务的Encoder-Decoder框架
  - 提出了C4数据集，包含750G高质量英文数据
  - 每项任务（包括翻译、问答和分类）都被视为将模型文本作为输入并训练它生成一些目标文本（Text-to-Text 任务）



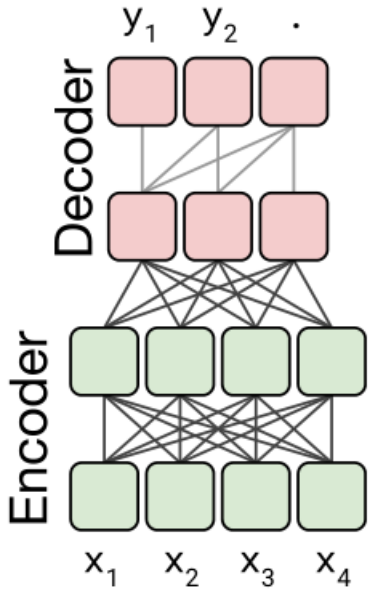
补充：序列到序列预训练语言模型T5

- T5模型的模型架构类型选择
  - Encoder-Decoder结构（传统的Transformer结构）
  - Decoder结构：（GPT的结构）
  - Prefix LM结构（UniLM的结构）
- 原因
  - 相较于BERT等，T5是一个生成式模型，框架一定要包含Decoder结构

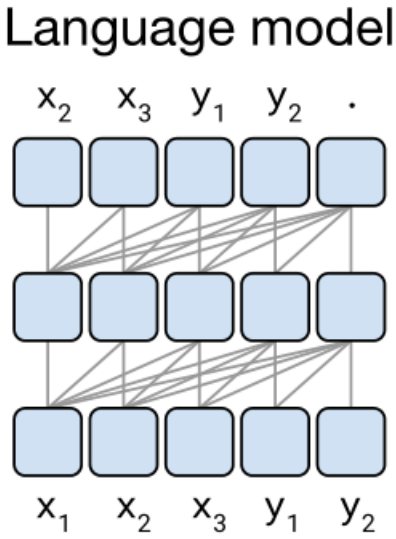


补充：序列到序列预训练语言模型T5

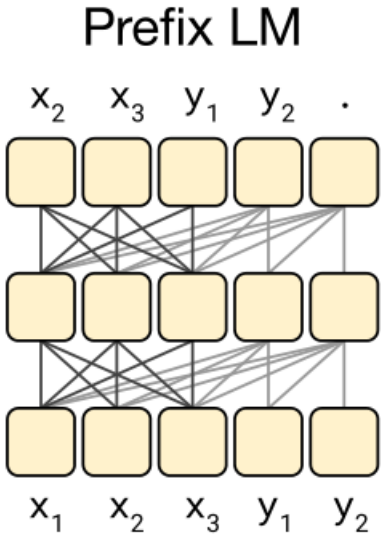
- 三种备选模型架构类型图解



Encoder-Decoder结构



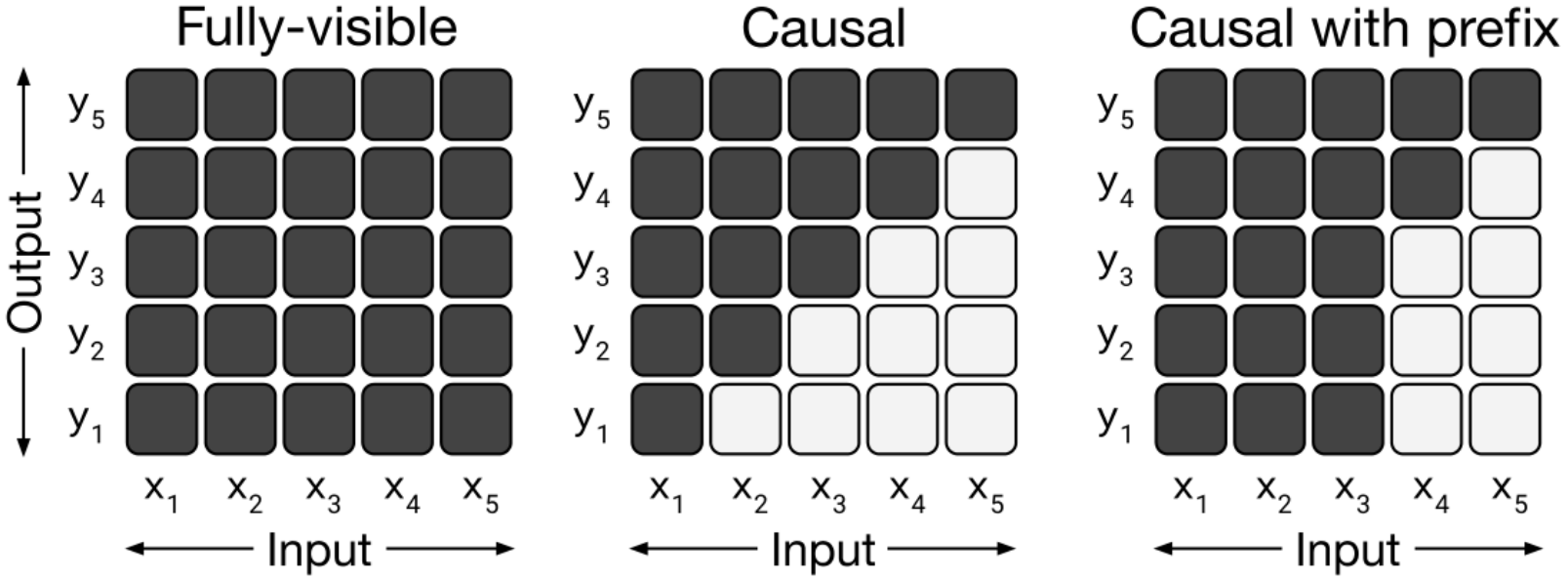
Decoder结构



Prefix LM结构

补充：序列到序列预训练语言模型T5

- 三种模型框架注意力机制MASK对象的转变



Encoder-Decoder结构　　Decoder结构　　Prefix LM结构

补充：序列到序列预训练语言模型T5

- T5模型框架的确定

Architecture	Objective	Params	Cost	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Encoder-decoder	Denoising	$2P$	M	**83.28**	**19.24**	**80.88**	**71.36**	**26.98**	**39.82**	**27.65**
Enc-dec, shared	Denoising	P	M	82.81	18.78	**80.63**	**70.73**	26.72	39.03	**27.46**
Enc-dec, 6 layers	Denoising	P	$M/2$	80.88	18.97	77.59	68.42	26.38	38.40	26.95
Language model	Denoising	P	M	74.70	17.93	61.14	55.02	25.09	35.28	25.86
Prefix LM	Denoising	P	M	81.82	18.61	78.94	68.11	26.43	37.98	27.39
Encoder-decoder	LM	$2P$	M	79.56	18.59	76.02	64.29	26.27	39.17	26.86
Enc-dec, shared	LM	P	M	79.60	18.13	76.35	63.50	26.62	39.17	27.05
Enc-dec, 6 layers	LM	P	$M/2$	78.67	18.26	75.32	64.06	26.13	38.42	26.89
Language model	LM	P	M	73.78	17.54	53.81	56.51	25.23	34.31	25.38
Prefix LM	LM	P	M	79.68	17.84	76.87	64.86	26.28	37.51	26.76

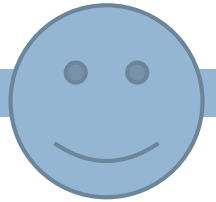
参数量

计算量

- 说明：
- P：指12层Transformer的参数量
- M：使用编码器解码器模型处理序列需要的FLOPs
- FLOPs：是每秒所执行的浮点运算次数

- 控制计算成本实验的直观实验
- Encoder-decoder结构性能胜出

补充：序列到序列预训练语言模型T5

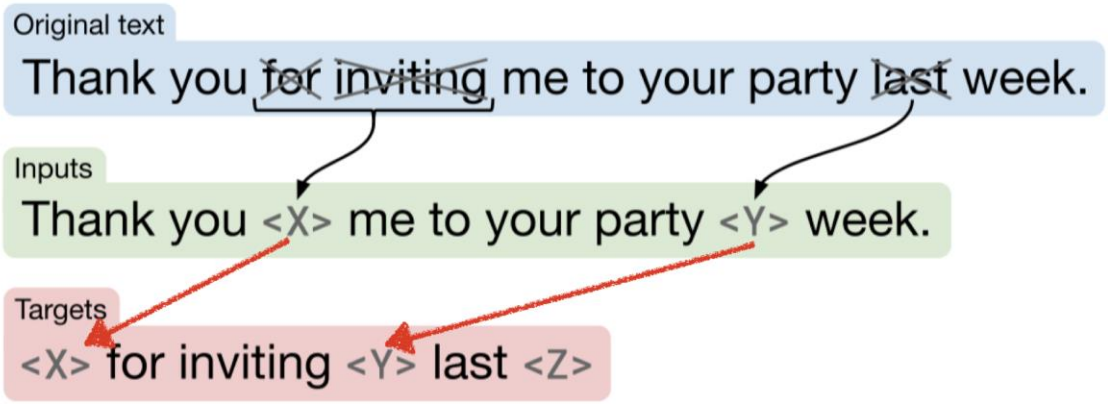


- T5模型训练任务形式的确定
  - 不同训练任务的输入和目标

Objective	Inputs	Targets
Prefix language modeling	Thank you for inviting	me to your party last week .
BERT-style Devlin et al. (2018)	Thank you <M> <M> me to your party apple week .	*(original text)*
Deshuffling	party me for your to . last fun you inviting week Thank	*(original text)*
MASS-style Song et al. (2019)	Thank you <M> <M> me to your party <M> week .	*(original text)*
I.i.d. noise, replace spans	Thank you <X> me to your party <Y> week .	<X> for inviting <Y> last <Z>
I.i.d. noise, drop tokens	Thank you me to your party week .	for inviting last
Random spans	Thank you <X> to <Y> week .	<X> for inviting me <Y> your party last <Z>

补充：序列到序列预训练语言模型T5

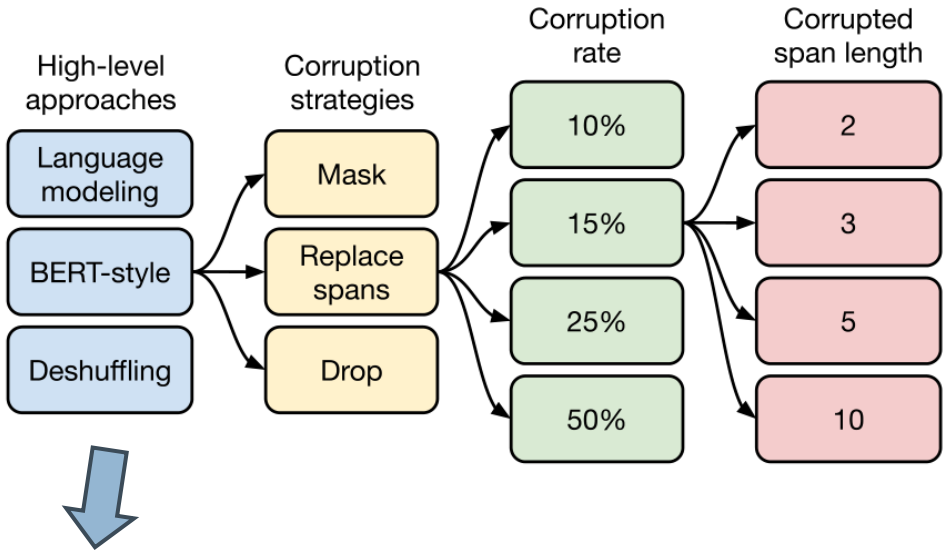
- **T5: Text-to-Text Transfer Transformer**
  - Transformer Encoder-Decoder 模型
  - BERT-style 式的破坏方法
  - 提出了一种基于span-corruption的无监督训练任务



  - 输入：对句子中的若干文本片段进行mask
  - 输出：预测被mask的文本片段

补充：序列到序列预训练语言模型T5

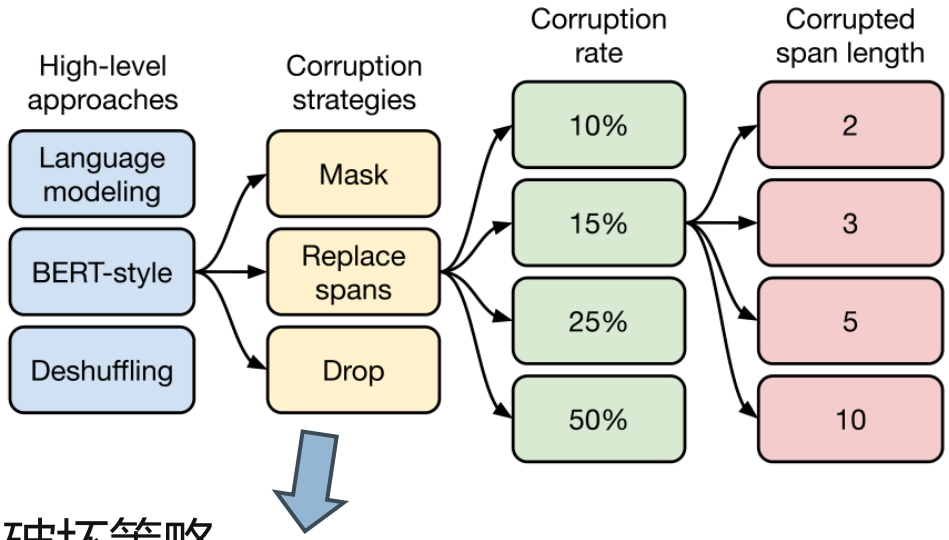
- T5模型训练任务形式的确定
  - 四方面对比实验流程



  - 第一方面：自监督方法
    - Language modeling式：和GPT-2 一样，从左到右预测
    - BERT-style 式：像 BERT 一样，将一部分给破坏掉，然后还原
    - Deshuffling （顺序还原）式：将文本打乱，然后进行还原

补充：序列到序列预训练语言模型T5

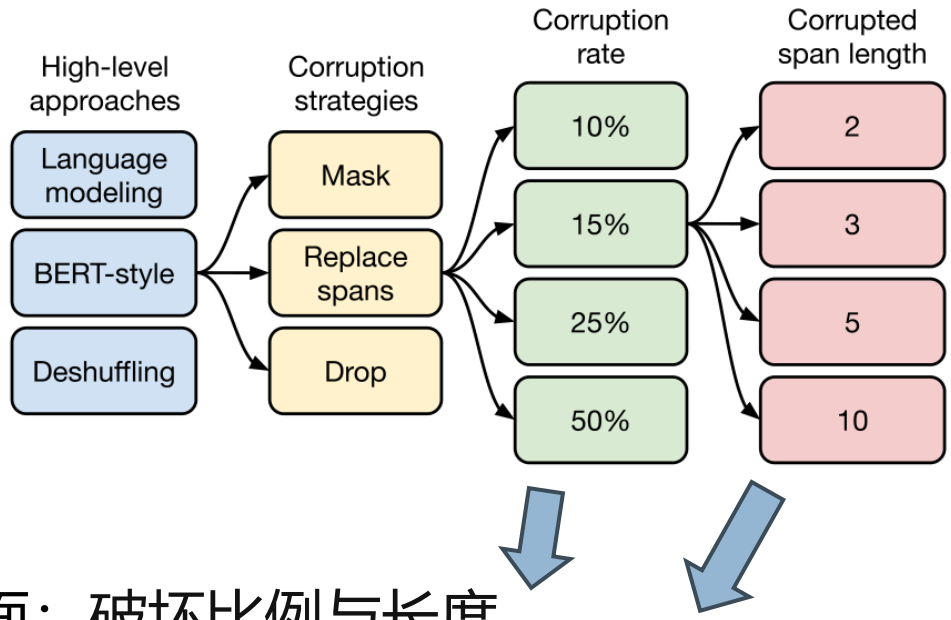
- T5模型训练任务形式的确定
  - 四方面对比实验流程



  - 第二方面：破坏策略
    - MASK式：随机破坏，将破坏的token更换为MASK
    - Replace span 式：破坏一小段，将一小段token更换为MASK标签，区别在于破坏邻近token
    - Drop式：将文本中一部分token直接丢弃，不进行替换

补充：序列到序列预训练语言模型T5

- T5模型训练任务形式的确定
  - 四方面对比实验流程



  - 第三四方面：破坏比例与长度
    - 更换破坏的比例以及这一小段要破坏文本的长度

预训练语言模型的应用

- 预训练
  - 有效学习语言的通用语义表示
- 微调（Fine-tuning）
  - 将通用表示应用于具体任务
  - 保留通用语言知识的基础上，针对目标任务的特性进行适配
  - 更好地满足下游任务需求
- 与传统方法中需要为每个任务精心设计特征并深入了解任务细节的方式相比，预训练语言模型大大简化了这一过程
- 只需将下游任务的输入输出形式转换为模型可处理的格式，便可通过微调获得性能优异的结果

预训练语言模型的应用

- 单句文本分类（Bert为例，下同）
  - 判断一段文本所属的类别
  - 例如，判断一段电影评价的情感倾向是正面还是负面，判断一篇新闻所属的类别
  - 步骤：
    - 使用 WordPiece 进行分词，得到分词后的句子$w_1 w_2 \ldots w_n$
    - 分别添加特殊字符 [CLS]和 [SEP]到句首和句尾
    - 经过输入层将其转换为 BERT 编码层所需的输入表示

$$w = [\text{CLS}] w_1 w_2 \ldots w_n, [\text{SEP}]$$

$$\mathbf{v} = \text{InputLayer}(w)$$

预训练语言模型的应用

- 单句文本分类
  - 步骤：
    - 输入表示进入编码层，在最后一层得到具备丰富上下文信息的表示
    - 使用 [CLS] 位置的隐藏层表示$h_{[CLS]}$进行预测
    - 在编码层之后，模型通过一个全连接层预测输入文本对应的类别

$$\mathbf{H} = \text{BERT}(\mathbf{V})$$

$$P = \text{Softmax}(\mathbf{h}_{[\text{CLS}]}\mathbf{W} + \mathbf{b})$$

    - 其中，$W \in R^{d \times K}$和$b \in R^{K}$分别为全连接层的权重和偏置，K为类别总数

预训练语言模型的应用

- 单句文本分类
  - 得到概率P后：
    - 若为训练阶段，可以计算P与真实标签间的交叉熵对模型参数进行训练
    - 若为预测阶段，可以取概率最高的一项作为输入文本的类别
- 句子对分类
  - 预测一对有关联的句子的类别
  - 例如判断一个句子的意思是否蕴含在另一个句子之中（即自然语言推理任务，Natural Language Inference，NLI）
  - 与单句分类的区别：处理的输入不同
  - BERT 处理这两个任务时，也主要在输入上有所区别

预训练语言模型的应用

- 句子对分类
  - 对于分词后的两个句子$w_1^{(1)}, w_2^{(1)}, \ldots, w_n^{(1)}$和$w_1^{(2)}, w_2^{(2)}, \ldots, w_m^{(2)}$，BERT 使用 [SEP] 作为分隔符，将两个句子拼接到一起，在输入层转换为输入表示：

$$w = [\text{CLS}] w_1^{(1)} w_2^{(1)} \ldots w_n^{(1)} [\text{SEP}] w_1^{(2)} w_2^{(2)} \ldots w_m^{(2)} [\text{SEP}]$$

$$\mathbf{v} = \text{InputLayer}(w)$$

  - 其中，$v \in R^{(n+m+3)\times d}$，$d$为模型隐藏层维度，$n$和$m$分别代表第一个句子和第二个句子的长度
  - 得到输入表示后，剩下的流程与单句文本分类一致

预训练语言模型的应用

- 序列标注
  - 解决的是字符级别或词级别的分类问题
  - 自然语言处理领域的基础任务
  - 例如分词、词性标注和命名实体识别（NER）

- 使用 BERT 等基于子词的预训练模型
  - 输入文本会通过模型自带的分词器切分为更小粒度的子词（subwords）
  - 破坏了序列标注中词和标签的一对一对应关系
  - 需要额外的处理方法，比如：
    - 所有子词共享原标签
    - 仅用第一个子词

预训练语言模型的应用

- 完成分词后
  - 将输入序列送入输入层转化为词向量
  - 将词向量送入预训练模型得到最终的隐藏层表示

$$w = [\text{CLS}]\, w_1^{(1)}\, w_2^{(1)} \ldots w_n^{(1)}\, [\text{SEP}]$$

$$\mathbf{v} = \text{InputLayer}(w)$$

$$\mathbf{h} = \text{BERT}(\mathbf{v})$$

  - 其中，$v \in R^{(n+2)\times d}$为模型输入层的输出，$h \in R^{(n+2)\times d}$为预训练模型最后一层的隐藏层表示，n为分词后的序列长度，d为模型隐藏层维度

预训练语言模型的应用

- 得到隐藏层表示后：
  - 使用一个分类器预测每个词在标签集上的概率分布

$$P = \text{Softmax}\left(\mathbf{h}_i \mathbf{W} + \mathbf{b}\right)$$

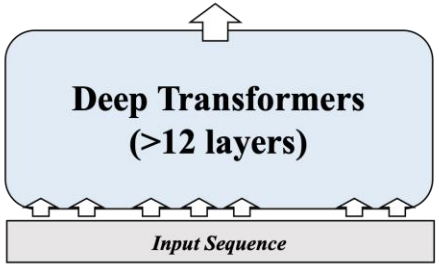
  - 其中，h_i是隐藏层表示h在第i时刻的分量，$i \in \{1, \ldots, n\}$
  - 使用交叉熵损失学习模型参数
  - 使用条件随机场等方法进一步提升序列标注性能

结语：预训练语言模型三要素

大数据
（无标注文本）

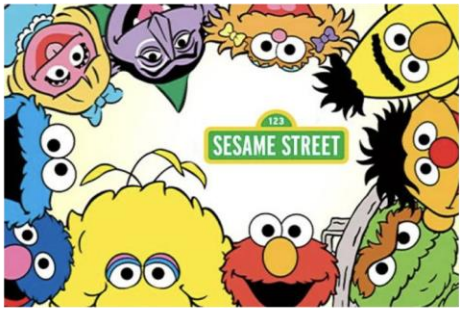


大模型
（深度神经网络）



大算力
（并行计算集群）





OpenAI

GPT

GPT-2

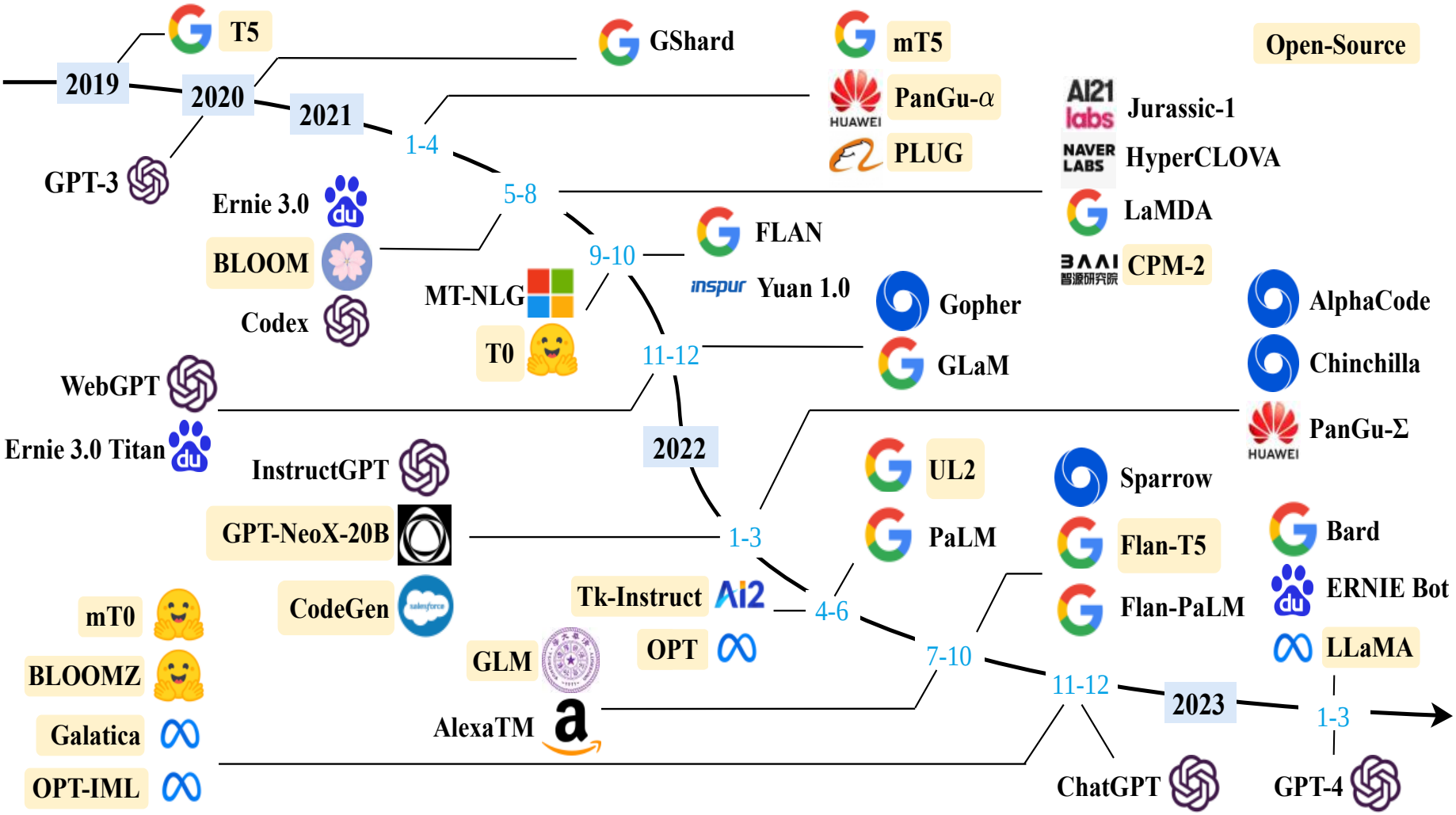
GPT-3

以及更多钱



结语：大模型时代到来




2019
2020
2021
2022
2023
T5
GPT-3
GShard
mT5
PanGu-α
PLUG
Open-Source
AI21 labs
Jurassic-1
NAVER LABS
HyperCLOVA
LaMDA
BAAI 智源研究院
CPM-2
1-4
5-8
9-10
11-12
1-3
4-6
7-10
11-12
1-3
Ernie 3.0
BLOOM
Codex
MT-NLG
T0
FLAN
inspur
Yuan 1.0
Gopher
GLaM
AlphaCode
Chinchilla
PanGu-Σ
HUAWEI
WebGPT
Ernie 3.0 Titan
InstructGPT
GPT-NeoX-20B
CodeGen
UL2
PaLM
Sparrow
Flan-T5
Flan-PaLM
Bard
ERNIE Bot
LLaMA
mT0
BLOOMZ
Galatica
OPT-IML
Tk-Instruct
AI2
OPT
GLM
AlexaTM
ChatGPT
GPT-4

习题

习题1：请描述分布式假设及其对于文本分布式表示的意义。

习题2：请简述Word2vec两种模型的核心思想。

习题3：在学习单词表示时，有哪些方法可以缓解未登录词的问题？

习题4：怎么评价词向量模型的优劣？

习题

习题5：请描述语言模型的目标。

习题6：相比于文本（句子或篇章）的分布式表示，利用语言模型对其建模的主要优点是什么？

习题7：对于给定词序列$w_1 w_2 \ldots w_m$，语言模型旨在建模其联合概率$\mathrm{P}(w_1 w_2 \ldots w_m)$，能不能直接计算？如果不能，为什么？有什么方法缓解吗？

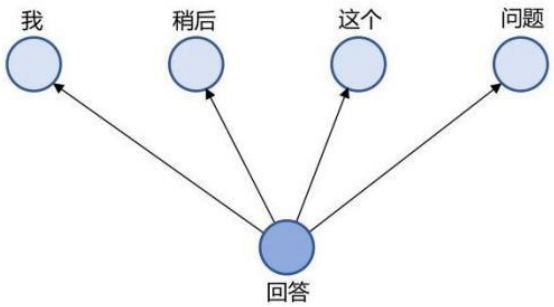
习题8：请简述循环神经网络语言模型的优点。

习题

习题9：有哪几类预训练语言模型？它们的优缺点是什么？

习题10：请简述BERT模型的训练过程。

习题11：下图展示了Skip-Gram模型的基本思想：给定当前词，模型旨在优化词向量，进而预测其上下文的单词。



习题

1) 假定我们的词汇表中有 100 个不同的单词，窗口大小设置为 2（包括左侧的两个上下文单词和右侧的两个上下文单词）。要计算图中显示的所有条件概率，最少需要执行多少次向量点积运算?

2) 在上述问题中，最耗时的操作是什么？如何解决这个问题？请展示你的计算过程。访问单词的嵌入不需要任何计算。